Міністерство освіти і науки України
Дрогобицький державний педагогічний університет імені Івана Франка
Кафедра фізики та інформаційних систем

«До захисту допускаю»
Завідувач кафедри фізики та інформаційних технологій
кандидат фізико-математичних наук, доцент
_____Віталій Гольський
«_____»_____2025 р.

Розробка системи веб-аналітики та візуалізації історії браузерної активності користувача

Спеціальність: 122 Комп'ютерні науки

Магістерська робота

на здобуття кваліфікації – магістр з комп'ютерних наук

Автор роботи: Михайлівський Я.О.
_____
підпис

Науковий керівник : доктор фізико-математичних наук
Столярчук І. Д.
_____
підпис

Дрогобич, 2025

ЗАХИСТ МАГІСТЕРСЬКОЇ РОБОТИ

Розробка системи веб-аналітики та візуалізації історії браузерної активності користувача

Оцінка за стобальною шкалою: _____
Оцінка за національною чотирибальною шкалою: _____

Коротка мотивація захисту:

_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____

_____ Голова ЕК _____ _____
дата підпис прізвище, ім'я

Секретар ЕК _____ _____
підпис прізвище, ім'я

Анотація

Михайлівський Я. О. Розробка системи веб-аналітики та візуалізації історії браузерної активності користувача. Магістерська робота, Дрогобицький державний педагогічний університет імені Івана Франка, Дрогобич, 2025 р.

Подана магістерська робота присвячена розробці системи веб-аналітики та візуалізації браузерної активності користувача під назвою «Chartex» – інструменту контролю цифрової активності, що допомагає оцінювати онлайн-трафік і час, витрачений на веб-ресурси. На відміну від існуючих аналогів, система забезпечує повний захист приватності користувача завдяки реалізації Zero Knowledge архітектури. Всі приватні дані шифруються на стороні клієнта за допомогою AES-256 перед відправкою на сервер , а ключі шифрування зберігаються лише локально. Розробка включала реалізацію трирівневої архітектури: Chrome Extension (для збору та шифрування даних), Backend API (на Node.js та MongoDB для зберігання зашифрованих записів) та Frontend Dashboard (Angular 19 з ApexCharts для візуалізації). Система надає детальну інтерактивну візуалізацію активності, включно з тепловими картами та круговими діаграмами, а також підтримує розширену систему фільтрації та пошуку по даних. Впровадження системи сприятиме підвищенню самодисципліни користувачів та ефективному тайм-менеджменту.

Ключові слова: система аналітики браузерної активності, Zero Knowledge архітектура, шифрування AES-256, Angular, Node.js, MongoDB, розширена фільтрація.

Abstract

Mykhailivskyi Y. O. Development of a web analytics system for visualizing browser activity history. Master's thesis, Drohobych Ivan Franko State Pedagogical University, Drohobych, 2025.

The submitted master's thesis focuses on the development of a web analytics and visualization system for browser activity named «Chartex» – a digital activity tracking tool that helps assess online traffic and time spent on web resources. Unlike existing counterparts, the system ensures full user privacy through a Zero Knowledge architecture. All private data is encrypted on the client side with AES- 256 before being sent to the server, while encryption keys remain stored locally. The project implements a three-tier architecture: a Chrome Extension (for data collection and encryption), a Backend API (Node.js and MongoDB for storing encrypted records), and a Frontend Dashboard (Angular 19 with ApexCharts for visualization). The system offers detailed interactive visualizations, including heat maps and pie charts, and supports advanced filtering and data search. Its adoption fosters greater user self-discipline and more effective time management.Keywords: browser activity analytics system, Zero Knowledge architecture, AES-256 encryption, Angular, Node.js, MongoDB, advanced filtering.

Keywords: browser activity analytics system, Zero Knowledge architecture, AES-256 encryption, Angular, Node.js, MongoDB, advanced filtering.Let me know if you need any further tweaks or a localized version.

Зміст

ВСТУП 9
1. Актуальність 9
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ 11
1.1. Аналіз існуючих рішень 11
1.1.1. Браузерна вбудована історія 11
1.1.2. Сторонні сервіси аналітики 11
1.1.3. Висновки з аналізу 11
1.2. Вимоги до системи 12
1.2.1. Функціональні вимоги 12
1.2.2. Нефункціональні вимоги 13
1.3. Технічне завдання 13
1.3.1. Опис системи 13
1.3.2. Технологічний стек 14
1.3.3. Обґрунтування вибору технологічного стеку Backend 15
1.3.4. Обґрунтування архітектури Frontend 15
1.3.5. Інструменти збору даних, криптографія та автентифікація 16
1.3.6. Обґрунтування вибору інструментів візуалізації 17
1.3.7. Основні функції 17
РОЗДІЛ 2. АРХІТЕКТУРА ТА ПРОЕКТУВАННЯ СИСТЕМИ 19
2.1. Загальна архітектура системи 19
2.2. Потік даних та реалізація принципу Zero Knowledge 20

2.3. Архітектура Backend 21
2.3.1. Теоретичні основи Backend архітектури 22
2.3.2. Основні компоненти 23
2.3.3. API Endpoints 23
2.3.4. Автентифікація 24
2.4. Архітектура Frontend 24
2.4.1. Теоретичні основи Frontend архітектури 24
2.4.2. Структура додатку 26
2.4.3. Основні компоненти 26
2.4.4. Сервіси 29
2.5. Архітектура Chrome Extension 29
2.5.1. Теоретичні основи Chrome Extensions 29
2.5.2. Компоненти Extension 30
2.5.3. Збір даних 31
2.5.4. Шифрування в Extension 33
2.6. Моделі даних 34
2.6.1. User (Користувач) 35
2.6.2. HistoryItem (Запис історії) 36
2.7. Архітектура безпеки та шифрування 38
2.7.1. Теоретичні основи Zero Knowledge Architecture 38
2.7.2. Детальний процес шифрування 40
2.7.3. Хешування для анонімної статистики 43
2.7.4. Автентифікація та авторизація 44

2.7.5. GDPR Compliance 45
РОЗДІЛ 3. РЕАЛІЗАЦІЯ СИСТЕМИ 47
3.1. Реалізація Backend 47
3.1.1. Структура проекту 47
3.1.2. Головний файл сервера 47
3.1.3. Реалізація автентифікації 50
3.1.4. API Endpoints – Історія 51
3.1.5. API Endpoints – Користувачі 52
3.2. Реалізація Frontend 52
3.2.1. Архітектурні принципи реалізації 52
3.2.2. Компонент Dashboard 53
3.2.3. Компоненти графіків 55
3.2.4. Компонент Settings 60
3.2.5. Сервіс ApiService 60
3.3. Реалізація Chrome Extension 61
3.3.1. Теоретичні аспекти реалізації Extension 61
3.3.2. Background Script (Service Worker) 62
3.3.3. Реалізація шифрування в Extension 63
3.3.4. Оптимізація відправки даних 64
3.3.5. Реалізація автентифікації в Extension 65
3.4. Реалізація візуалізації даних 67
3.4.3. Алгоритм категорізації сайтів 69
РОЗДІЛ 4. ТЕСТУВАННЯ ТА РОЗГОРТАННЯ 72

4.1. Стратегія тестування 72
4.1.1. Теоретичні основи тестування 72
4.1.2. Типи тестування 73
4.1.3. Тестування вручну 74
4.2. Інструкції з встановлення 75
4.2.1. Вимоги до системи 75
4.2.2. Встановлення Backend 76
4.2.3. Встановлення Frontend 77
4.2.4. Встановлення Extension 78
4.3. Інструкції з налаштування 78
4.3.1. Налаштування Firebase 79
4.3.2. Налаштування MongoDB 79
4.3.3. Налаштування CORS 80
4.3.4. Налаштування Extension 80
4.4. Розгортання 81
ВИСНОВКИ 83
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 86

ВСТУП

1. Актуальність

У сучасному цифровому світі користувачі проводять більшість часу в інтернеті, використовуючи веб-браузери для роботи, навчання, розваг та спілкування. Браузерна історія містить величезну кількість цінної інформації про поведінку користувача, його інтереси, звички та патерни активності. Однак, існуючі рішення для аналітики браузерної активності мають серйозні обмеження, включаючи **проблеми з приватністю** (більшість сервісів зберігають дані у відкритому вигляді) , **відсутність контролю** над власними даними та **обмежену функціональність**, що надає лише базову статистику без детальної візуалізації. Система **Chartex** розроблена для вирішення цих проблем, надаючи користувачам повний контроль над своїми даними через концепцію **Zero Knowledge [7]** з шифруванням на стороні клієнта, детальну аналітику та візуалізацію браузерної активності.

Метою роботи є розробка системи аналітики браузерної активності з **повним захистом приватності користувача**, що дозволяє збирати, зберігати, аналізувати та візуалізувати дані про відвідування веб-сайтів з дотриманням принципів **Zero Knowledge архітектури**.

Для досягнення цієї мети необхідно було вирішити наступні сім задач : провести аналіз існуючих рішень для аналітики браузерної активності та визначити їх недоліки; розробити архітектуру системи з підтримкою Zero Knowledge шифрування; реалізувати Backend компонент для зберігання зашифрованих даних та обробки запитів; реалізувати Frontend компонент для візуалізації та аналітики даних; реалізувати Chrome Extension [2] для автоматичного збору даних про браузерну активність; забезпечити безпеку даних через шифрування на клієнті; та розробити інтерфейс для детальної візуалізації аналітики.

Об’єктом дослідження є процеси збору, зберігання, обробки та візуалізації даних про браузерну активність користувача. **Предметом дослідження** є методологія та технології розробки системи аналітики браузерної активності з захистом приватності на основі **Zero Knowledge архітектури**.

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

1.1. Аналіз існуючих рішень

1.1.1. Браузерна вбудована історія

Сучасні браузери надають базову функціональність перегляду історії, однак мають суттєві обмеження:

- Відсутність аналітики та статистики
- Обмежена пошукова функціональність
- Відсутність візуалізації даних

1.1.2. Сторонні сервіси аналітики

Існуючі сторонні сервіси, такі як RescueTime та Toggl Track, хоча і надають більш розширену функціональність, все ж мають значні недоліки. Серед них — проблеми з приватністю, оскільки дані зберігаються на серверах постачальників у відкритому вигляді. Це також призводить до відсутності повного контролю над даними, оскільки користувач залежить від постачальника сервісу. Крім того, не всі сервіси дотримуються вимог щодо захисту даних, що ставить під сумнів їхню відповідність GDPR compliance. Нарешті, спостерігається обмежена кастомізація, через що користувач не може повністю налаштувати аналітику під свої специфічні потреби.

1.1.3. Висновки з аналізу

На основі проведеного аналізу визначено основні вимоги до системи:

1. **Приватність**: всі приватні дані повинні шифруватися на стороні клієнта
2. **Контроль**: користувач повинен мати повний контроль над своїми даними
3. **Функціональність**: система повинна надавати детальну аналітику та візуалізацію

4. **GDPR compliance**: дотримання вимог щодо захисту персональних даних
5. **Продуктивність**: система повинна працювати швидко та ефективно

1.2. Вимоги до системи

1.2.1. Функціональні вимоги

1. Збір даних про браузерну активність
   - Система повинна автоматично збирати дані про відвідування веб-сайтів.
   - Збір повинен відбуватися в фоновому режимі без втручання користувача.
   - Система повинна зберігати метадані про кожне відвідування (час, тривалість, URL, заголовок).
2. Шифрування даних
   - Всі приватні дані повинні шифруватися на стороні клієнта перед відправкою на сервер.
   - Ключі шифрування повинні зберігатися тільки на клієнті.
   - Сервер не повинен мати доступу до розшифрованих даних.
3. Візуалізація даних
   - Система повинна надавати різні типи графіків та діаграм
   - Графіки повинні відображати активність за різні періоди (день, тиждень, місяць, рік).
   - Система повинна підтримувати фільтрацію та пошук по даних.
4. Налаштування
   - Користувач повинен мати можливість налаштувати параметри збору даних.
   - Система повинна підтримувати різні мови інтерфейсу.
   - Система повинна підтримувати світлу та темну теми.

1.2.2. Нефункціональні вимоги

1. Безпека
    - Використання AES-256 [9] шифрування для захисту даних
    - Реалізація Zero Knowledge архітектури
    - Дотримання вимог GDPR
2. Продуктивність
    - Система повинна працювати без уповільнення роботи браузера
    - Візуалізація даних повинна відображатися швидко
    - Оптимізація запитів до бази даних
3. Надійність
    - Система повинна працювати без уповільнення роботи браузера
    - Візуалізація даних повинна відображатися швидко
    - Оптимізація запитів до бази даних
4. Масштабованість
    - Система повинна підтримувати велику кількість користувачів
    - База даних повинна бути оптимізована для швидкого пошуку
    - API повинен підтримувати пагінацію для великих обсягів даних

1.3. Технічне завдання

1.3.1. Опис системи

Chartex - це система аналітики браузерної активності, що складається з трьох основних компонентів:

1. **Chrome Extension** - розширення для браузера Chrome, що збирає дані про відвідування сайтів.
2. **Backend API** - серверна частина для зберігання зашифрованих даних та обробки запитів.
3. **Frontend Dashboard** - веб-інтерфейс для візуалізації та аналітики даних.

1.3.2. Технологічний стек

Backend:

- Node.js [10] - серверна платформа
- Express.js [3] - веб-фреймворк
- MongoDB [5] - база даних NoSQL
- Mongoose [15] - ODM для MongoDB
- Firebase Admin SDK [4] - автентифікація користувачів

Frontend:

- Angular 19 [1] - SPA framework
- Angular Material [11] - UI компоненти
- ApexCharts [6] - бібліотека для графіків
- RxJS [13] - реактивне програмування
- ngx-translate [14] - інтернаціоналізація

Extension:

- Chrome Extension Manifest V3
- Service Worker для фонового виконання
- Chrome Identity API для авторизації
- Chrome History API для доступу до історії

Шифрування:

- Crypto-JS [12] - бібліотека для шифрування на клієнті

- Node.js crypto module - шифрування на сервері

1.3.3. Обґрунтування вибору технологічного стеку Backend

Backend-частина системи Chartex реалізована на базі платформи Node.js у поєднанні з фреймворком Express.js та документо-орієнтованою базою даних MongoDB. Такий вибір забезпечує високу продуктивність та гнучкість, необхідні для роботи з великими обсягами асинхронних записів історії [5].

Node.js було обрано завдяки його подійно-орієнтованій архітектурі (event-driven architecture) та неблокуючому I/O (non-blocking I/O). Це дозволяє ефективно обробляти велику кількість одночасних з'єднань, що є критичним для застосунку з безперервним потоком даних від Chrome Extension [10].

Як мінімалістичний веб-фреймворк для Node.js, Express.js спрощує обробку HTTP-запитів, маршрутизацію та реалізацію RESTful API. Це дозволило швидко створити набір API Endpoints для операцій з ресурсами (отримання, додавання, видалення історії та налаштувань користувача) [3].

Для зберігання даних використовується MongoDB, документо-орієнтована NoSQL база даних. Основною перевагою є її гнучка схема, що дозволяє зберігати дані різної структури в одній колекції у форматі BSON. Це ідеально підходить для структури HistoryItem, яка містить як фіксовані метадані, так і динамічні зашифровані дані (encryptedData). Для зручності роботи з базою даних використано ODM Mongoose, що надає можливість визначення схем, валідації та middleware.

1.3.4. Обґрунтування архітектури Frontend

Для реалізації клієнтського веб-інтерфейсу, який є односторінковим додатком (SPA), було обрано фреймворк Angular 19. Використання Component-Based Architecture (архітектури, що базується на компонентах) дозволило створити модульну та легко підтримувану структуру коду. Зокрема, застосування Standalone

Components спростило організацію коду, усуваючи необхідність використання NgModule.

Динамічність та висока інтерактивність дашборду досягнуті завдяки широкому використанню бібліотеки RxJS (Reactive Extensions for JavaScript). RxJS забезпечує елегантну обробку асинхронних операцій, включаючи HTTP-запити до Backend API та управління станом фільтрів через об’єкти Observable [1]. Крім того, застосування бібліотеки Angular Material прискорило розробку та гарантувало консистентний вигляд елементів інтерфейсу [11].

1.3.5. Інструменти збору даних, криптографія та автентифікація

Система збору даних реалізована через Chrome Extension , розроблений на основі архітектури Manifest V3. Використання V3 є обов'язковим для підвищення безпеки та продуктивності і базується на моделі Service Worker для фонового виконання з доступом до API браузера (History, Storage, Identity) .

Центральним елементом безпеки є Zero Knowledge Architecture , яка гарантує, що сервер не має доступу до розшифрованих даних користувача. Для цього всі приватні дані шифруються на стороні клієнта за допомогою симетричного алгоритму AES (Advanced Encryption Standard). Було обрано режим роботи GCM (Galois/Counter Mode) , який, окрім конфіденційності, забезпечує автентичність даних за допомогою 256-бітного ключа [9].

Автентифікація користувачів здійснюється за допомогою сервісу Firebase Authentication , що надає готову інфраструктуру для безпечного входу через Google OAuth 2.0. Для авторизації запитів до Backend API використовуються JWT (JSON Web Tokens), які підтверджують автентичність користувача.

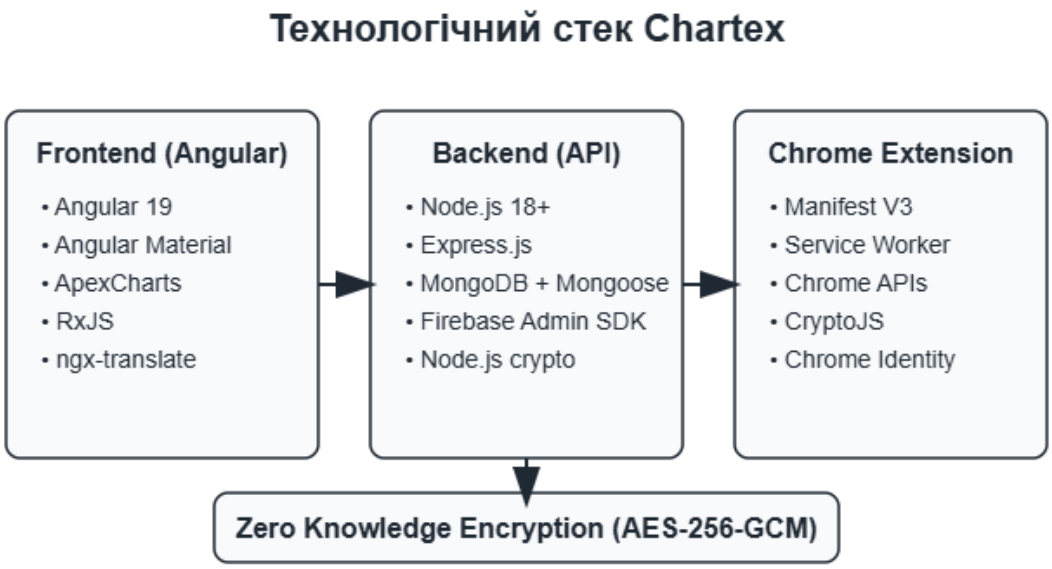


Рис. 1.3.5 - Архітектура технологічного стеку Chartex

1.3.6. Обґрунтування вибору інструментів візуалізації

Візуалізація даних відіграє ключову роль у системі аналітики Chartex, дозволяючи користувачам швидко та інтуїтивно сприймати великі обсяги інформації.

Для реалізації інтерактивних графіків було обрано бібліотеку ApexCharts. Її перевагами є широкий спектр типів візуалізації (лінійні, теплові карти, кругові діаграми) , висока інтерактивність (підтримка zoom, pan, tooltip) та продуктивність завдяки використанню SVG. Це дозволяє створювати адаптивний дашборд із підтримкою світлої та темної теми.

1.3.7. Основні функції

1. Автоматичний збір історії браузера
2. Шифрування приватних даних перед відправкою на сервер
3. Візуалізація активності через різні типи графіків:
   - Графік активності за днями
   - Теплова карта активності

- Кругова діаграма розподілу по категоріях
- Бульбашкова діаграма сайтів
- Топ найпопулярніших сайтів

4. Фільтрація та пошук по даних
5. Налаштування параметрів збору та відображення даних

РОЗДІЛ 2. АРХІТЕКТУРА ТА ПРОЕКТУВАННЯ СИСТЕМИ

2.1. Загальна архітектура системи

Система **Chartex** побудована на основі **трирівневої архітектури**, яка забезпечує модульність, масштабованість та безпеку через чітке **розділення відповідальностей** між компонентами. Ця структура відповідає принципу єдиної відповідальності, де кожен модуль виконує лише одну, чітко визначену функцію.

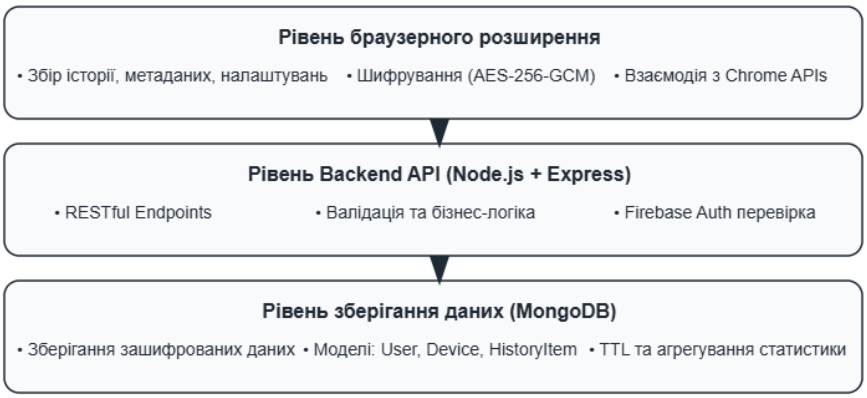


Рис. 2.1.1 - Загальна архітектура системи Chartex

Основні компоненти системи та їхні функції:

1. Chrome Extension: Виконує роль агента збору даних. Він безперервно працює у фоні браузера користувача, збирає дані про активність, виконує шифрування приватних даних (URL, заголовок) на стороні клієнта за допомогою алгоритму AES-256-GCM та відправляє їх на сервер.

2. Backend API: Реалізований як центральне сховище даних на базі Node.js/Express.js та MongoDB. Його функція — обробка запитів, валідація та зберігання зашифрованих записів. Backend виконує лише роль посередника і, завдяки архітектурі Zero Knowledge, не має доступу до ключів шифрування та, відповідно, до розшифрованих даних користувача.

3. Frontend Dashboard: Веб-інтерфейс, розроблений на Angular, який відповідає за відображення та взаємодію з користувачем. Його завдання — завантаження зашифрованих даних із Backend, їхнє дешифрування на стороні клієнта та відображення у вигляді інтерактивних графіків та діаграм.

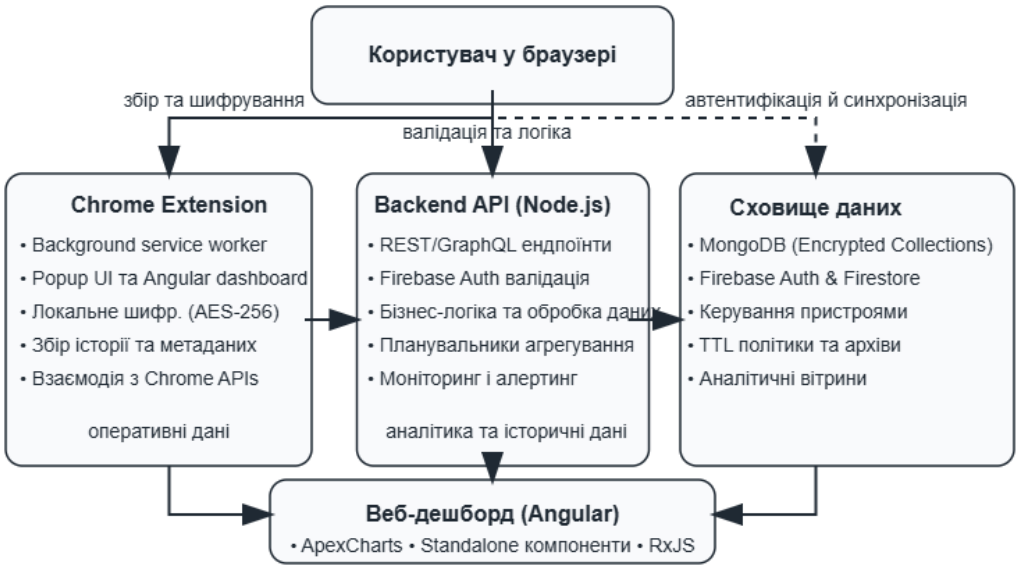


Рис. 2.1.2 - Компоненти системи та їх взаємодія

2.2. Потік даних та реалізація принципу Zero Knowledge

Потік даних у системі Chartex чітко розділений на етапи: збір, шифрування, зберігання та відображення.

На етапі збору та шифрування Chrome Extension збирає метадані (час, тривалість, домен) та приватні дані (повний URL, заголовок сторінки). Приватні дані негайно шифруються на клієнті, використовуючи унікальний ключ користувача та алгоритм AES-256-GCM, що реалізує принцип Zero Knowledge. Створюється об’єкт, який містить зашифровані дані (ciphertext), ініціалізаційний вектор (IV) та аутентифікаційний тег (Auth Tag) [9].

На етапі зберігання цей об'єкт відправляється на Backend API через захищене з’єднання HTTPS. Сервер валідує запит, але ніколи не намагається розшифрувати дані. Вони зберігаються у MongoDB в зашифрованому вигляді, що унеможливлює доступ до конфіденційної інформації навіть у разі компрометації сервера.

На етапі відображення Frontend Dashboard запитує зашифровані записи через API. Для розшифрування він використовує ключ, який зберігається у локальному сховищі браузера користувача. Після успішного дешифрування дані обробляються, агрегуються та відображаються у вигляді інтерактивних графіків за допомогою бібліотеки ApexCharts, забезпечуючи детальний аналіз браузерної активності.



Рис. 2.2 - Діаграма потоку даних в системі

2.3. Архітектура Backend

Backend побудований на Node.js та Express.js і виконує наступні функції. Він реалізує RESTful API архітектуру, що забезпечує масштабованість та гнучкість системи.

2.3.1. Теоретичні основи Backend архітектури

RESTful API архітектура передбачає використання стандартних HTTP методів та статус-кодів для операцій з ресурсами. Це робить API інтуїтивним та легким для розуміння та використання.

Middleware архітектура в Express дозволяє створювати ланцюжки обробників, які виконуються послідовно. Це дозволяє винести спільну логіку (автентифікація, логування, обробка помилок) в окремі middleware функції.

Модель MVC (Model-View-Controller) частково застосована в архітектурі Backend, хоча це є API-сервер. Роль Models виконують Mongoose схеми, які слугують для представлення даних. Controllers реалізовані як обробники маршрутів, що містять основну бізнес-логіку системи. Компонент представлення у даній архітектурі відсутній, оскільки Backend не генерує HTML-сторінки, а лише відповідає клієнту даними у форматі JSON.

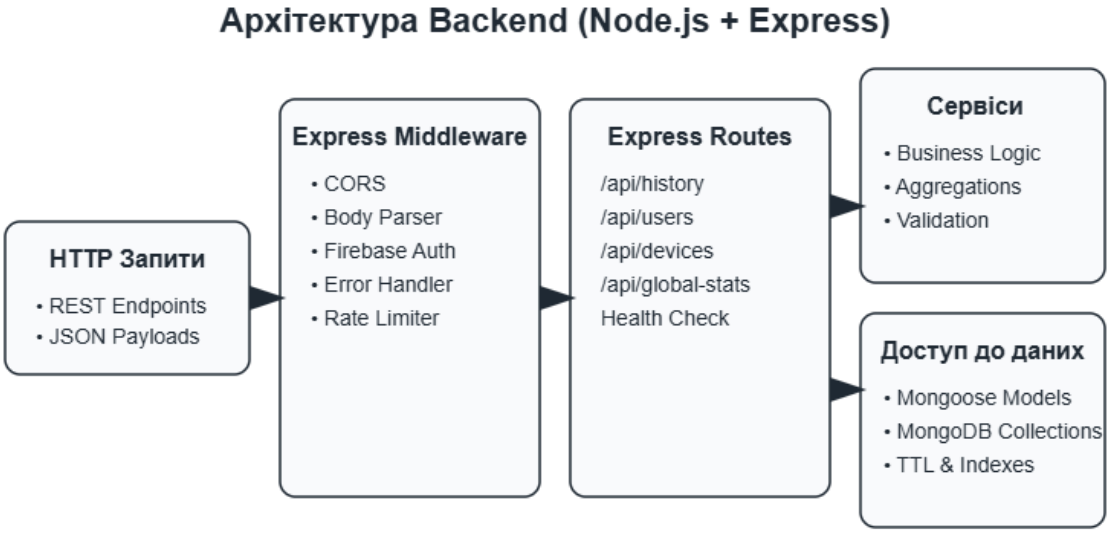


Рис. 2.3.1 - Архітектура Backend API

Backend побудований на Node.js та Express.js і виконує наступні функції:

2.3.2. Основні компоненти

API Routes:

- /api/history - управління історією відвідувань
- /api/users - управління користувачами
- /api/devices - управління пристроями
- /api/global-stats - анонімна глобальна статистика

Моделі даних:

- User - інформація про користувача
- Device - інформація про пристрій
- HistoryItem - запис про відвідування сайту

Утиліти:

- auth.js - автентифікація через Firebase
- crypto.js - функції шифрування (для серверної обробки при потребі)

2.3.3. API Endpoints

Історія (/api/history):

- POST /add - додати один запис історії
- POST /batch - додати кілька записів одночасно
- GET / - отримати історію з фільтрами
- GET /stats - отримати статистику історії
- DELETE / - видалити записи історії

- DELETE /clear-all - очистити всю історію користувача

Користувачі (/api/users):

- POST /register - реєстрація/оновлення користувача
- GET /profile - отримати профіль користувача
- PUT /settings - оновити налаштування користувача
- GET /stats - отримати статистику користувача
- DELETE /account - видалити акаунт користувача
- POST /clear-data - очистити дані користувача

2.3.4. Автентифікація

Всі захищені endpoints вимагають JWT токен від Firebase у заголовку Authorization: Bearer <token>. Middleware authenticateToken перевіряє валідність токена через Firebase Admin SDK.

2.4. Архітектура Frontend

Frontend реалізований на Angular 19 з використанням standalone компонентів. Це сучасний підхід до розробки Angular додатків, який дозволяє створювати самодостатні компоненти без необхідності оголошення в NgModule.

2.4.1. Теоретичні основи Frontend архітектури

Component-Based Architecture - архітектурний підхід, де додаток розділяється на незалежні, повторно використовувані компоненти. Кожен компонент інкапсулює свою логіку та шаблон, що спрощує розробку та тестування.

Dependency Injection (DI) - це патерн проектування, який дозволяє компонентам отримувати залежності (сервіси) через конструктор, замість

створення їх безпосередньо. Angular має вбудовану систему DI, яка автоматично надає потрібні залежності.

Reactive Programming з RxJS дозволяє працювати з асинхронними потоками даних за допомогою Observable. Це дозволяє елегантно обробляти події, HTTP-запити, таймери та інші асинхронні операції.

Change Detection - механізм Angular для відстеження змін у даних та оновлення DOM. Angular використовує стратегію OnPush для оптимізації продуктивності, де компонент оновлюється тільки при зміні вхідних параметрів або подіях [1].

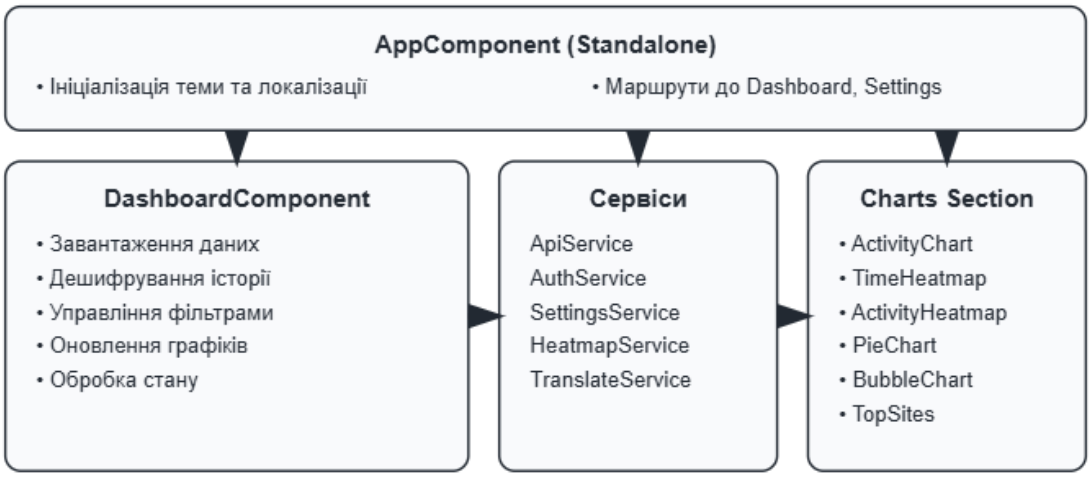


Рис. 2.4.1 - Архітектура Backend API

2.4.2. Структура додатку

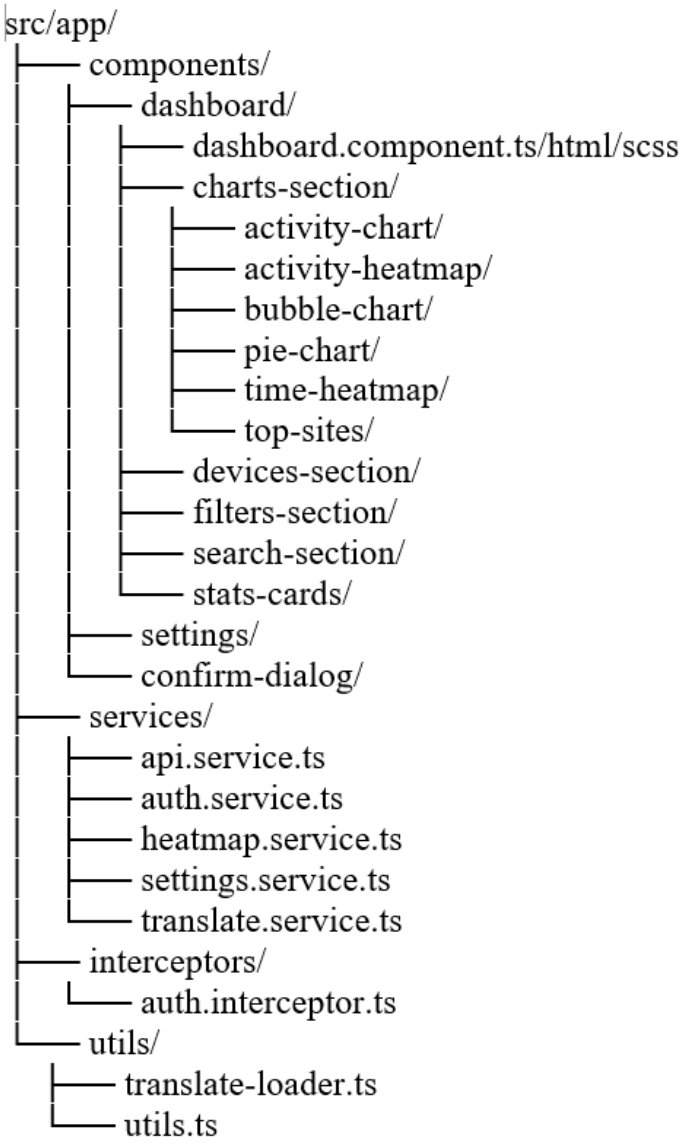


Рис. 2.4.2 - Структура додатку

2.4.3. Основні компоненти

Dashboard Component є центральним елементом інтерфейсу, що виконує роль головного компонента, який координує роботу всіх секцій. Він відповідає за керування загальним станом додатку та координує роботу всіх дочірніх компонентів. Для оптимізації продуктивності Dashboard Component використовує стратегію *ChangeDetectionStrategy.OnPush*. Його основні обов'язки включають завантаження даних з API через сервіси, дешифрування отриманих даних за

допомогою ключа з локального сховища, обробку фільтрів та пошуку, управління станом додатку (зокрема, вибраний період та пристрій), а також координацію оновлення всіх дочірніх компонентів.

Dashboard містить секцію графіків (Charts Section), яка складається з декількох дочірніх компонентів, що візуалізують дані. Серед них: Activity Chart (лінійний графік активності за днями), Activity Heatmap (теплова карта активності по днях тижня), Time Heatmap (активність по годинах по тижнях), Pie Chart (кругова діаграма розподілу по категоріях), Bubble Chart (візуалізація сайтів за часом проти кількості відвідувань) та Top Sites (список найбільш популярних сайтів).

Крім того, інтерфейс має Filters Section для гнучкого управління відображенням даних, що дозволяє користувачу здійснити вибір періоду (7 днів, 30 днів, 90 днів, 1 рік або весь час), вибрати конкретний пристрій та застосувати розширені фільтри (за категоріями, тривалістю чи діапазоном дат). Додатково реалізовано Search Section для швидкого пошуку по сайтах.

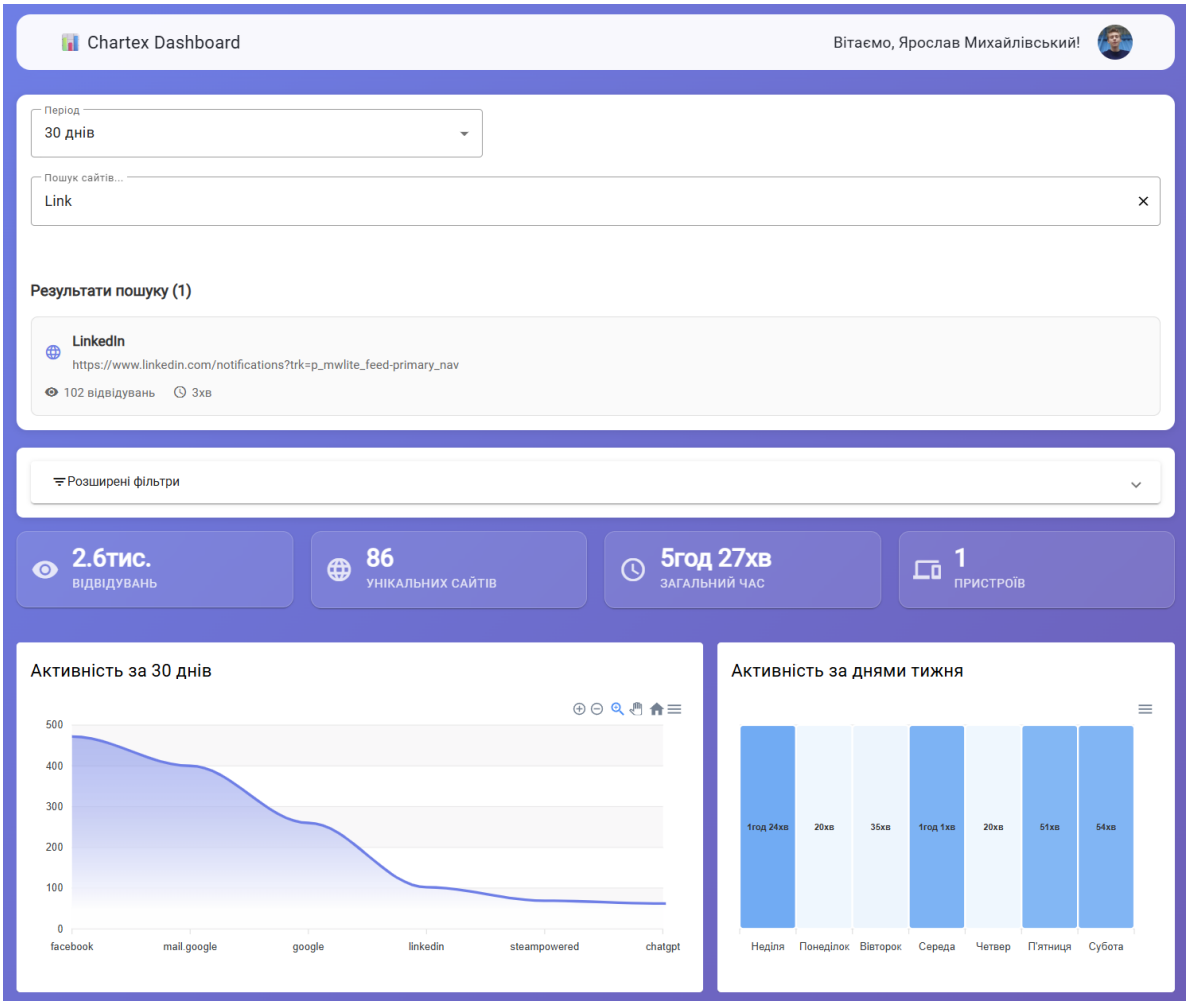


Рис. 2.4.3.1 - Інтерфейс Dashboard з графіками

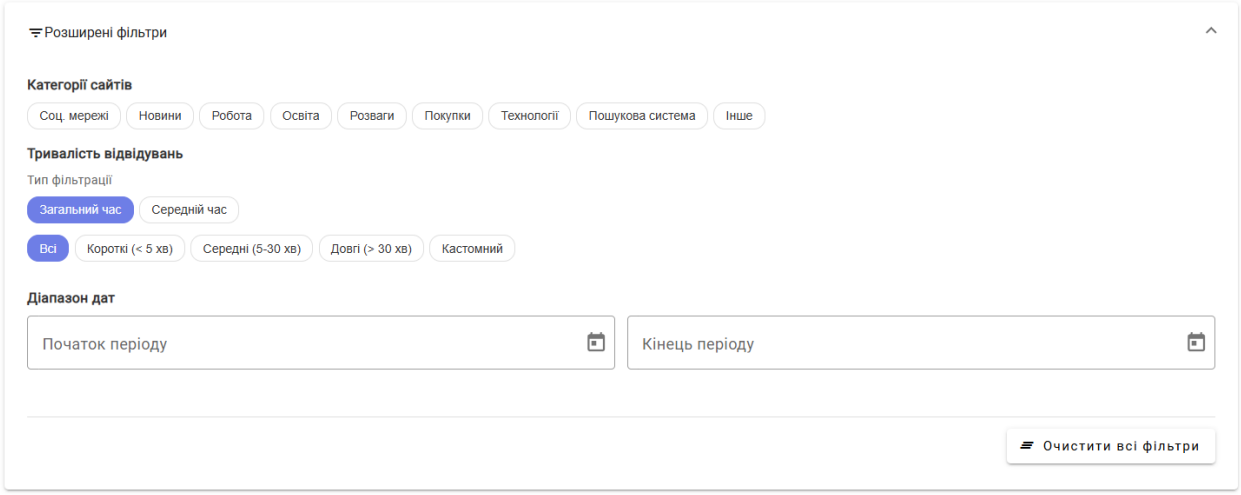


Рис. 2.4.3.2 – Інтерфейс розширеного пошуку

2.4.4. Сервіси

ApiService — це сервіс, призначений для взаємодії з Backend API. Він реалізує патерн Repository для інкапсуляції логіки доступу до даних. Для виконання HTTP-запитів та обробки асинхронних операцій сервіс використовує HttpClient та RxJS Observable. Основні функції сервісу включають надання методів для отримання історії, статистики та інформації про пристрої; він також відповідає за дешифрування даних на стороні клієнта, управління ключем шифрування та обробку помилок з повторними спробами при збоях.

AuthService — це сервіс автентифікації, який інтегрується з Firebase Authentication. Він використовує концепцію Reactive Programming для постійного відстеження стану автентифікації користувача. Серед його ключових функцій — забезпечення входу через Google (Firebase Auth), управління сесією користувача, отримання ключа шифрування з Chrome Extension, а також вихід та очищення сесії.

HeatmapService є спеціалізованим сервісом, який бере на себе завдання обробки сирих даних історії та їхнє перетворення у формат, зручний для подальшого відображення у вигляді теплових карт. **SettingsService**, у свою чергу, відповідає за управління налаштуваннями користувача, зберігаючи їх локально та синхронізуючи з Backend за потреби.

2.5. Архітектура Chrome Extension

Extension реалізований на Manifest V3 з використанням Service Worker. Manifest V3 впроваджено Google для підвищення безпеки та продуктивності розширень, замінивши застарілий Manifest V2.

2.5.1. Теоретичні основи Chrome Extensions

Chrome Extensions API надає широкий спектр можливостей для розширення функціональності браузера: доступ до історії браузера через chrome.history;

зберігання даних через chrome.storage; взаємодія з вкладками через chrome.tabs; втентифікація через chrome.identity [2].

Service Worker в Extensions працює інакше, ніж у звичайних веб-додатках. Він активується при встановленні розширення та реагує на події, такі як повідомлення від інших частин розширення або зовнішні події браузера.

Content Security Policy (CSP) в Manifest V3 значно посилюється. Заборонено використання eval(), вбудованих скриптів через onclick тощо. Це збільшує безпеку, але вимагає адаптації підходів до розробки.

Message Passing - механізм комунікації між різними частинами розширення. Service Worker може обмінюватися повідомленнями з Content Scripts, Popup та іншими компонентами через chrome.runtime.sendMessage().

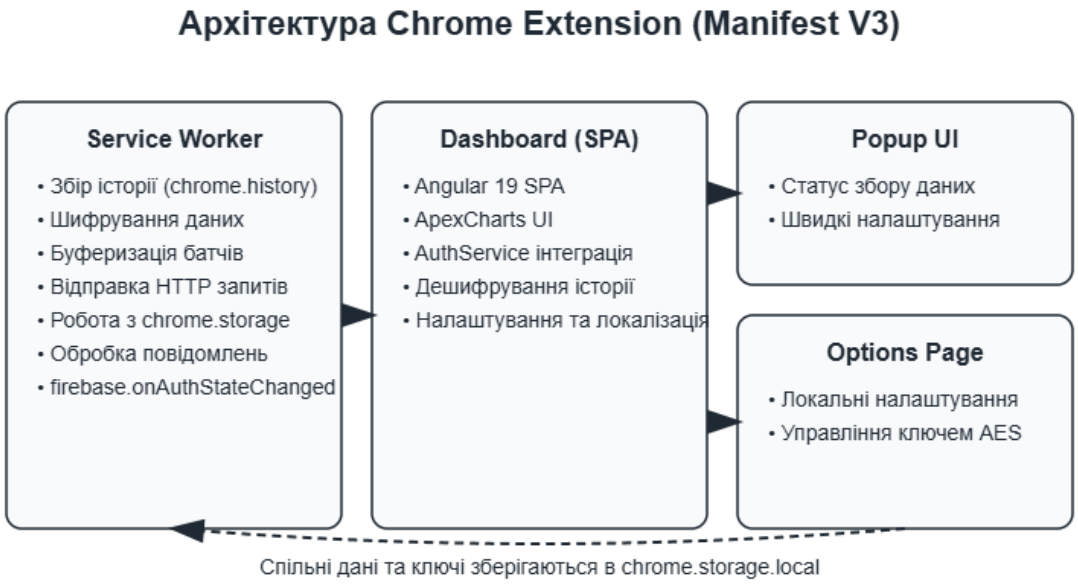


Рис. 2.5.1 - Архітектура Chrome Extension

2.5.2. Компоненти Extension

Background Script (background.js) - головний скрипт Extension. Включає збір історії через Chrome History API, шифрування даних перед відправкою,

відправку даних на Backend, автентифікація через Firebase, управління сесією та deviceId.

Popup (popup.html, popup.js) - інтерфейс для швидкого доступу, що включає статус збору даних, швидкі налаштування та посилання на Dashboard.

Dashboard - повноцінний Angular SPA, вбудований в Extension. Він відкривається через chrome-extension:// протокол та має повний доступ до всіх функцій системи.

2.5.3. Збір даних

Extension збирає дані про браузерну активність через Chrome History API, який надає доступ до історії відвідувань користувача. Збір даних відбувається в реальному часі через обробку подій браузера.

Chrome History API надає кілька методів для роботи з історією. Один з основних це chrome.history.onVisited - подія, яка спрацьовує при відвідуванні сторінки, також важливим є chrome.history.onVisitRemoved - подія, яка спрацьовує при видаленні запису з історі. При розширеному пошуку особливо допоміжним був chrome.history.search() - метод для пошуку в історії за різними критеріями. І не менш цінний з усіх chrome.history.getVisits() – служить для отримання всіх відвідувань конкретного URL.

Обробка події onVisited:

```
chrome.history.onVisited.addListener((historyItem) => {
  // historyItem містить:
  // - url: URL сторінки
  // - title: заголовок сторінки
  // - lastVisitTime: час останнього відвідування
  // - visitCount: кількість відвідувань
```

```
  // - typedCount: кількість разів введено вручну
});
```

Зібрані дані про браузерну активність включають детальну інформацію про відвідування веб-сайтів. До них належать: **URL сторінки** (повна адреса відвіданої сторінки), **Заголовок сторінки** (title елемент HTML сторінки) та **Час відвідування** (точний час у мілісекундах, або *timestamp*). Також фіксується **Тривалість перебування**, яка розраховується на основі різниці часу між відвідуваннями (якщо доступна). Система записує **Тип переходу**, що може бути *typed* (введено вручну), *link* (перехід по посиланню), *auto_bookmark* (автоматичне відкриття закладки) чи *reload* (оновлення сторінки). Крім того, збирається **Referrer** — URL сторінки, з якої був здійснений перехід (якщо доступний), а також **Метадані пристрою**, що охоплюють *userAgent*, роздільність екрану, операційну систему та версію браузера.

Для забезпечення продуктивності та зменшення навантаження на мережу застосовується низка методів оптимізації збору даних. Використовується буфер для накопичення записів перед їхньою відправкою, а також батч-відправка даних, що значно зменшує кількість необхідних HTTP-запитів. Додатково застосовується фільтрація дублікатів на рівні Extension та дебаунсинг для уникнення надто частих оновлень, що підвищує ефективність процесу.



Рис. 2.5.3 - Схема збору даних Extension

2.5.4. Шифрування в Extension

Шифрування даних в Extension є критично важливим етапом для забезпечення приватності користувача. Всі приватні дані шифруються на стороні Extension перед відправкою на сервер, забезпечуючи реалізацію Zero Knowledge архітектури.

Процес шифрування включає наступні кроки:

1. Генерація або отримання ключа шифрування
    - При першому запуску Extension генерується унікальний ключ
    - Ключ зберігається в chrome.storage.local для постійного доступу
    - Ключ ніколи не передається на сервер
2. Підготовка даних до шифрування
    - Створюється об’єкт з приватними даними (URL, title, referrer, etc.)
    - Дані серіалізуються в JSON формат
    - Додаються метадані для ідентифікації
3. Шифрування об’єкта
    - Використовується AES шифрування через CryptoUtils
    - Дані кодуються в Base64 для безпечної передачі
    - Результат шифрування містить всю необхідну інформацію для дешифрування
4. Створення анонімних метаданих
    - Створюється хеш домену для анонімної статистики
    - Хеш генерується за допомогою SHA-256 з сіллю
    - Хеш дозволяє агрегувати статистику без знання реальних доменів
5. Формування об’єкта для відправки
    - Об’єднуються зашифровані дані та метадані
    - Додаються технічні метадані (час, тип пристрою, браузер)

- Об’єкт готовий до відправки на сервер



Рис. 2.5.4 - Схема процесу шифрування

2.6. Моделі даних

Моделі даних визначають структуру інформації, яка зберігається в системі. Всі моделі реалізовані за допомогою Mongoose - ODM (Object Document Mapper) для MongoDB, що дозволяє визначати схеми, валідацію та взаємозв’язки між даними [15].

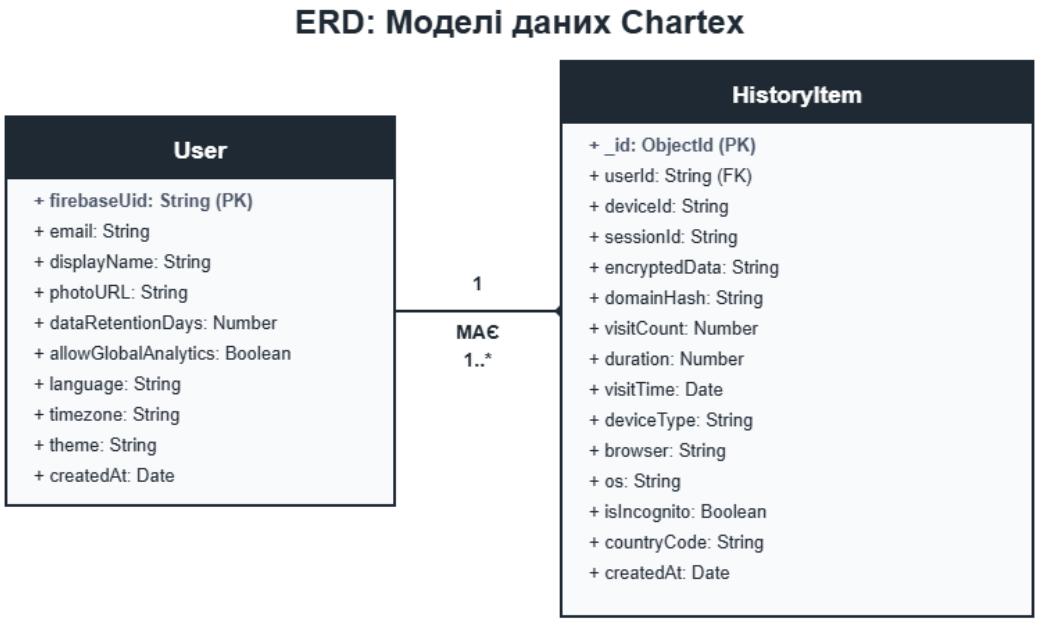


Рис. 2.6 - ER-діаграма моделей даних

2.6.1. User (Користувач)

```
{
 firebaseUid: String,          // Унікальний ID з Firebase
 email: String,                // Email користувача
 displayName: String,          // Відображаєме ім'я
 photoURL: String,             // URL фото профілю
 createdAt: Date,              // Дата створення
 lastLogin: Date,              // Останній вхід
 dataRetentionDays: Number,    // Днів зберігання даних
 allowGlobalAnalytics: Boolean, // Дозвіл на глобальну аналітику
 language: String,             // Мова інтерфейсу
 timezone: String,             // Часовий пояс
 theme: String                 // Тема (light/dark)
}
```

Модель User (Користувач) містить основну інформацію про користувача системи, включаючи firebaseUid — унікальний ідентифікатор користувача з Firebase, який використовується як первинний ключ (primary key) , а також email — адресу електронної пошти для ідентифікації та комунікації , displayName — відображуване ім’я користувача в інтерфейсі , та photoURL — URL фото профілю користувача. Додатково зберігаються налаштування приватності, зокрема dataRetentionDays — кількість днів зберігання даних (за замовчуванням 365) , та allowGlobalAnalytics — дозвіл на використання анонімних даних для глобальної статистики. Для оптимізації швидкого доступу та пошуку даних у моделі створюються індекси на полі firebaseUid та на полі email.

2.6.2. HistoryItem (Запис історії)

```
{
  userId: String,              // ID користувача
  deviceId: String,            // ID пристрою
  sessionId: String,           // ID сесії
  encryptedData: String,       // Зашифровані приватні дані
  domainHash: String,          // Хеш домену для анонімної статистики
  visitCount: Number,          // Кількість відвідувань
  typedCount: Number,          // Кількість разів введено вручну
  duration: Number,            // Тривалість (секунди)
  transitionType: String,      // Тип переходу
  deviceType: String,          // Тип пристрою
  browser: String,             // Браузер
  language: String,            // Мова
  screenResolution: String,    // Роздільність екрану
  os: String,                  // Операційна система
  visitTime: Date,             // Час відвідування
  createdAt: Date,             // Час створення запису
```

```
  updatedAt: Date,            // Час оновлення
  isBookmarked: Boolean,      // Додано в закладки
  isIncognito: Boolean,       // Режим інкогніто
  countryCode: String,        // Код країни
  timeOfDay: Number,          // Година дня (0-23)
  dayOfWeek: Number,          // День тижня (0-6)
  month: Number,              // Місяць (1-12)
  year: Number                // Рік
}
```

Приватні дані в encryptedData (зашифровані): дані в полі encryptedData зберігаються у зашифрованому вигляді та містять:

```
{
  url: String,                // URL сторінки
  title: String,              // Заголовок сторінки
  faviconUrl: String,         // URL іконки
  referrer: String,           // Посилання-джерело
  notes: String,              // Нотатки користувача
  tags: [String]              // Теги
}
```

Для забезпечення високої продуктивності та ефективного керування даними у моделі HistoryItem застосовується низка оптимізаційних заходів. Оптимізація моделі передбачає використання індексів на полях userId, deviceId, та visitTime для значного прискорення операцій швидкого пошуку. Крім того, застосовується TTL індекс (Time To Live) на полі createdAt для автоматичного видалення старих записів після закінчення встановленого терміну зберігання. Також створюється індекс на полі domainHash, необхідний для швидкої агрегації статистики.

Щодо забезпечення цілісності та коректності даних, виконується валідація даних. Вона включає обов’язкову перевірку наявності ключових полів, таких як userId, deviceId, encryptedData та domainHash. Додатково проводиться валідація формату дат, а також перевірка діапазонів числових значень, таких як тривалість (duration) та кількість відвідувань (visitCount).



Рис. 2.6.2 - Структура моделі HistoryItem

2.7. Архітектура безпеки та шифрування

Безпека та захист приватності є ключовими аспектами системи Chartex. Реалізація Zero Knowledge архітектури забезпечує максимальний захист даних користувача, навіть у випадку компрометації сервера.

2.7.1. Теоретичні основи Zero Knowledge Architecture

Zero Knowledge Architecture - це архітектурний підхід, при якому сервер не має доступу до розшифрованих даних користувача. Це досягається шляхом шифрування всіх приватних даних на клієнті перед їх передачею на сервер [7].

Переваги Zero Knowledge архітектури:

- Максимальна приватність: навіть якщо сервер буде скомпрометований, зловмисники не зможуть отримати доступ до приватних даних.
- Довіра користувача: користувач може бути впевнений, що його дані захищені.
- GDPR compliance: автоматичне дотримання вимог щодо захисту персональних даних.
- Юридична захищеність: постачальник сервісу не несе відповідальності за розкриття даних, оскільки не має доступу до них.

Основні принципи реалізації:

- Сервер ніколи не має доступу до розшифрованих даних
- Всі приватні дані шифруються на стороні клієнта перед відправкою
- Ключі шифрування зберігаються тільки на клієнті (в localStorage або chrome.storage.local)
- Сервер зберігає тільки зашифровані дані та анонімні метадані
- Неможливість відновлення даних без ключа шифрування

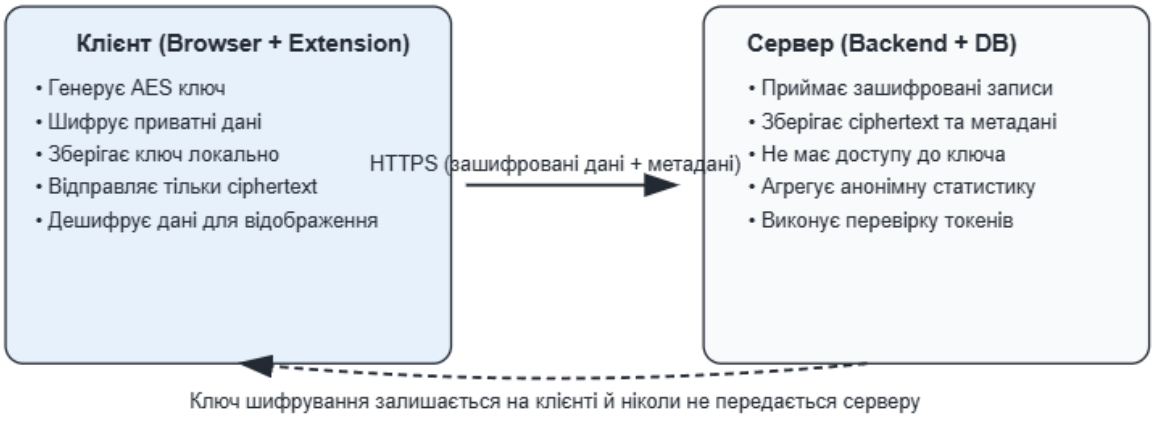


Рис. 2.7.1 - Схема Zero Knowledge архітектури

2.7.2. Детальний процес шифрування

Етап 1: Генерація та зберігання ключа

При першому запуску Extension або Frontend генерується криптографічно стійкий випадковий ключ шифрування. Генерація виконується за допомогою криптографічно безпечного генератора випадкових чисел (Crypto.getRandomValues()).

Ключ зберігається в chrome.storage.local для Extension (синхронізується між вкладками Extension) та localStorage для Frontend (зберігається локально в браузері).

Етап 2: Підготовка даних до шифрування

Перед відправкою даних на сервер виконується наступна послідовність дій:

1. **Створення об’єкта з приватними даними**
    - Формується об’єкт, що містить всі приватні дані (URL, title, referrer, notes, tags)
    - Об’єкт структурований для зручності обробки
1. **Серіалізація в JSON**
    - Об’єкт перетворюється в JSON рядок
    - Забезпечується коректна обробка Unicode символів
2. **Шифрування за допомогою AES**
    - Використовується алгоритм AES для шифрування
    - Генерується випадковий IV (Initialization Vector) для кожної операції шифрування
    - IV зберігається разом із зашифрованими даними для можливості дешифрування
3. **Кодування в Base64**

- Результат шифрування кодується в Base64 для безпечної передачі через HTTP
- Base64 забезпечує коректну передачу бінарних даних у текстових протоколах

Етап 3: Відправка на сервер

Зашифровані дані відправляються на сервер разом з:

- Метаданими (час, тривалість, тип переходу)
- Анонімними даними (хеш домену)
- Технічними метаданими (пристрій, браузер, OS)

Етап 4: Зберігання на сервері

На сервері виконується:

- Отримання зашифрованих даних
- Валідація структури даних (перевірка наявності обов'язкових полів)
- Валідація авторизації користувача
- Збереження в MongoDB без спроб розшифрування

Сервер не має можливості розшифрувати дані, оскільки не має доступу до ключа шифрування.

Етап 5: Дешифрування для відображення

При завантаженні даних для відображення виконується зворотна операція:

1. **Отримання зашифрованих даних з сервера**
   – Frontend завантажує дані через API
   – Дані приходять у зашифрованому вигляді

2. **Отримання ключа шифрування**
   - Ключ отримується з localStorage або chrome.storage.local
   - Якщо ключ не знайдено, дані не можуть бути розшифровані
3. **Декодування Base64**
   - Base64 рядок декодується в бінарні дані
4. **Розшифрування за допомогою AES**
   - Використовується той самий ключ та IV (який зберігався з зашифрованими даними)
   - Виконується розшифрування за алгоритмом AES
5. **Десеріалізація JSON**
   - Розшифрований рядок парситься як JSON
   - Відновлюється оригінальний об’єкт з приватними даними
6. **Використання для відображення**
   - Розшифровані дані використовуються для побудови графіків та відображення інформації

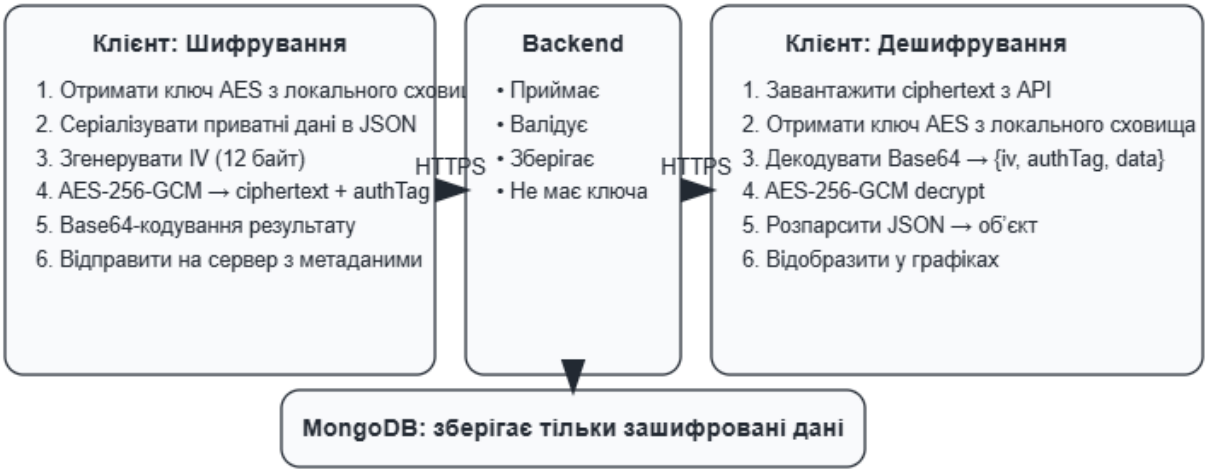


Рис. 2.7.2 - Схема процесу шифрування та дешифрування

2.7.3. Хешування для анонімної статистики

Для глобальної анонімної статистики використовується хешування доменів. Це дозволяє агрегувати статистику про популярність сайтів без зберігання реальних доменів, що забезпечує додатковий рівень приватності.

Процес хешування:

1. **Витягування домену з URL**
    1) З URL витягується домен (наприклад, з https://www.example.com/page витягується example.com)
    2) Видаляється протокол, www префікс та шлях
2. **Створення хешу**
    1) Використовується алгоритм SHA-256 для створення хешу
    2) До домену додається сіль (domain + 'chartex') для підвищення безпеки
    3) Результат - 64-символьний hex рядок
3. **Зберігання хешу**
    1) Хеш зберігається в полі domainHash разом із зашифрованими даними
    2) Хеш не може бути зворотно перетворений в домен
4. **Агрегація статистики**
    1) Сервер може агрегувати статистику по хешах
    2) Можна визначити популярність доменів без знання їх реальних назв
    3) Дані залишаються анонімними

Приклад хешування:

- Домен: example.com
- Із сіллю: example.comchartex

- SHA-256 хеш: a1b2c3d4e5f6... (64 символи)

Цей підхід забезпечує баланс між приватністю та можливістю отримання цінної анонімної статистики.



Рис. 2.7.3 - Схема хешування для анонімної статистики

2.7.4. Автентифікація та авторизація

Автентифікація реалізована через Firebase Authentication, який надає безпечну та масштабовану інфраструктуру для управління користувачами. Використання Firebase дозволяє не турбуватися про безпеку зберігання паролів, обробку токенів тощо [4].

Процес автентифікації:

1. **Вхід через Google OAuth**
   - Користувач натискає кнопку “Увійти через Google”
   - Extension або Frontend викликає Firebase Auth
   - Firebase перенаправляє користувача на сторінку входу Google
   - Після успішного входу Google повертає код авторизації
2. **Отримання JWT токена**

– Firebase обмінює код авторизації на idToken
– idToken містить інформацію про користувача та підпис Firebase
– Токен має обмежений термін дії (зазвичай 1 година)

3. **Використання токена для API**
   – Токен додається в заголовок Authorization: Bearer <token>
   – Кожен запит до API містить токен для авторизації
4. **Перевірка токена на Backend**
   – Middleware authenticateToken отримує токен з заголовка
   – Токен перевіряється через Firebase Admin SDK
   – Якщо токен валідний, запит обробляється далі
   – Якщо токен невалідний, повертається помилка 403

JWT структура: JWT токен складається з трьох частин, розділених крапками:

- **Header** - містить тип токена та алгоритм підпису
- **Payload** - містить дані про користувача (UID, email, exp тощо)
- **Signature** - підпис, який гарантує автентичність токена

Безпека токенів:

- Токени мають обмежений термін дії
- Токени підписуються приватним ключем Firebase
- Неможливо підробити токен без доступу до приватного ключа
- Старі токени автоматично стають невалідними

2.7.5. GDPR Compliance

Система Chartex повністю дотримується вимог GDPR (Загального регламенту про захист даних), надаючи користувачеві максимальний контроль над його

інформацією. Зокрема, користувач має повний контроль над своїми даними та має можливість видалення всіх даних одним запитом [8]. Також передбачене налаштування терміну зберігання даних та опціональна участь в глобальній аналітиці. Це забезпечується через прозорість щодо того, які дані збираються.

РОЗДІЛ 3. РЕАЛІЗАЦІЯ СИСТЕМИ

Розділ містить детальний опис реалізації всіх компонентів системи Chartex. Кожен компонент розглядається з точки зору архітектури, технологій та конкретної реалізації.

3.1. Реалізація Backend

Backend компонент реалізує серверну частину системи, що відповідає за зберігання даних, обробку запитів та надання API для клієнтських додатків. Реалізація базується на Node.js та Express.js фреймворку.

3.1.1. Структура проекту

```
backend/
├── src/
│   ├── index.js            # Головний файл сервера
│   ├── models/
│   │   ├── User.js         # Модель користувача
│   │   ├── Device.js       # Модель пристрою
│   │   └── HistoryItem.js  # Модель запису історії
│   ├── routes/
│   │   ├── history.js      # Маршрути історії
│   │   ├── users.js        # Маршрути користувачів
│   │   ├── devices.js      # Маршрути пристроїв
│   │   └── globalStats.js  # Глобальна статистика
│   └── utils/
│       ├── auth.js         # Автентифікація
│       └── crypto.js       # Утиліти шифрування
├── package.json
└── README.md
```

Рис. 3.1.1 - Схема хешування для анонімної статистики

3.1.2. Головний файл сервера

Головний файл сервера (src/index.js) відповідає за комплексну ініціалізацію всіх компонентів Backend системи Chartex.

Процес ініціалізації починається з Express додатку: створюється екземпляр Express, налаштовується *middleware* для коректної обробки JSON та CORS, а також налаштовується обробка помилок [3]. Після цього відбувається підключення до MongoDB, що включає встановлення з'єднання з базою даних, обробку можливих помилок підключення та налаштування необхідних опцій.

Далі виконується ініціалізація Firebase Admin SDK для налаштування автентифікації через Firebase та перевірки наявності необхідних облікових даних (*credentials*). Наступним кроком є підключення маршрутів, що передбачає реєстрацію всіх API endpoints системи та налаштування *middleware* для автентифікації запитів .

Завершальний етап — запуск сервера, під час якого починається прослуховування вказаного порту та логування статусу запуску системи .

Файл src/index.js ініціалізує Express сервер, підключає до MongoDB, налаштовує Firebase Admin SDK та підключає маршрути:

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const admin = require('firebase-admin');

const app = express();

// CORS налаштування
app.use(cors({
  origin: [
    'http://localhost:80',
    'http://localhost:8080',
```

```
    /^chrome-extension:\/\/.*$/
  ],
  credentials: true
}));

app.use(express.json());

// Підключення до MongoDB
mongoose.connect(process.env.MONGODB_URI);

// Ініціалізація Firebase Admin
admin.initializeApp({
  credential: admin.credential.cert({
    projectId: process.env.FIREBASE_PROJECT_ID,
    clientEmail: process.env.FIREBASE_CLIENT_EMAIL,
    privateKey: process.env.FIREBASE_PRIVATE_KEY
  })
});

// Підключення маршрутів
app.use('/api/history', historyRoutes);
app.use('/api/users', usersRoutes);
app.use('/api/devices', devicesRoutes);
app.use('/api/global-stats', globalStatsRoutes);

// Health check
app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', message: 'Chartex backend працює!' });
```

```
});

app.listen(process.env.PORT || 5000);
```

3.1.3. Реалізація автентифікації

Middleware authenticateToken в src/utils/auth.js реалізує механізм авторизації запитів до API. Middleware - це функція, яка виконується між отриманням запиту та викликом обробника маршруту.

Робота middleware authenticateToken реалізує механізм авторизації запитів до API, функціонуючи як проміжний обробник між отриманням запиту та викликом основного обробника маршруту.

Процес роботи middleware розпочинається з перевірки наявності токена в заголовку Authorization, після чого відбувається витягування токена із цього заголовку. Далі здійснюється перевірка валідності токена через Firebase Admin SDK. Якщо токен є валідним, інформація про автентифікованого користувача додається до об’єкта запиту , і управління передається наступному обробнику.

Під час обробки запиту також передбачена обробка помилок: якщо токен виявляється відсутнім, middleware повертає статус 401 Unauthorized; якщо ж токен є невалідним, повертається статус 403 Forbidden. У випадку, коли під час самої перевірки виникає внутрішня помилка, повертається статус 500 Internal Server Error.

Middleware authenticateToken в src/utils/auth.js перевіряє валідність Firebase JWT токенів:

```
const admin = require('firebase-admin');

async function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
```

```
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'Токен не надано' });
  }

  try {
    const decodedToken = await admin.auth().verifyIdToken(token);
    req.user = decodedToken;
    next();
  } catch (error) {
    res.status(403).json({ error: 'Невірний токен' });
  }
}

module.exports = { authenticateToken };
```

3.1.4. API Endpoints – Історія

Управління записами історії браузерної активності відбувається через низку RESTful кінцевих точок.

Для додавання записів реалізовано два методи: POST /api/history/add, який додає один запис історії, та POST /api/history/batch, який додає кілька записів одночасно, будучи оптимізованим для масового збору даних.

Для отримання даних використовуються методи GET: GET /api/history отримує історію з повною підтримкою фільтрів, таких як deviceId, from, to, search, limit та offset. Крім того, GET /api/history/stats надає агреговану статистику, що включає групування даних за днями, годинами або доменами.

Функціонал видалення включає метод DELETE /api/history, що дозволяє видаляти записи за заданими фільтрами, а також DELETE /api/history/clear-all, призначений для повного очищення всієї історії користувача.

3.1.5. API Endpoints – Користувачі

Керування профілем та даними користувача здійснюється через кілька кінцевих точок. Для реєстрації або оновлення профілю користувача використовується метод POST /api/users/register. Отримання профілю з відповідною статистикою відбувається через GET /api/users/profile , а отримання статистики користувача за вибраний період — через GET /api/users/stats. Оновлення налаштувань, таких як приватність (privacy), мова (language) та тема (theme), виконується за допомогою PUT /api/users/settings.

Для управління життєвим циклом даних передбачені два методи видалення: DELETE /api/users/account для видалення акаунту та всіх пов'язаних даних , а також POST /api/users/clear-data для очищення даних без видалення самого акаунту.

3.2. Реалізація Frontend

Frontend компонент реалізований на Angular 19 з використанням сучасних практик розробки, таких як standalone компоненти, реактивне програмування та оптимізація продуктивності через ChangeDetectionStrategy.OnPush.

3.2.1. Архітектурні принципи реалізації

Standalone Components - всі компоненти в проекті є standalone, що означає, що вони не потребують оголошення в NgModule. Це спрощує структуру проекту та робить код більш модульним.

OnPush Change Detection - використання стратегії OnPush для оптимізації продуктивності. Компонент оновлюється тільки коли: змінюються вхідні

параметри (@Input), виникають події всередині компонент, викликається markForCheck() або detectChanges() вручну [1].

Reactive Programming - широке використання RxJS для роботи з асинхронними операціями. Observable використовуються для HTTP-запитів, обробки подій, управління станом [13].

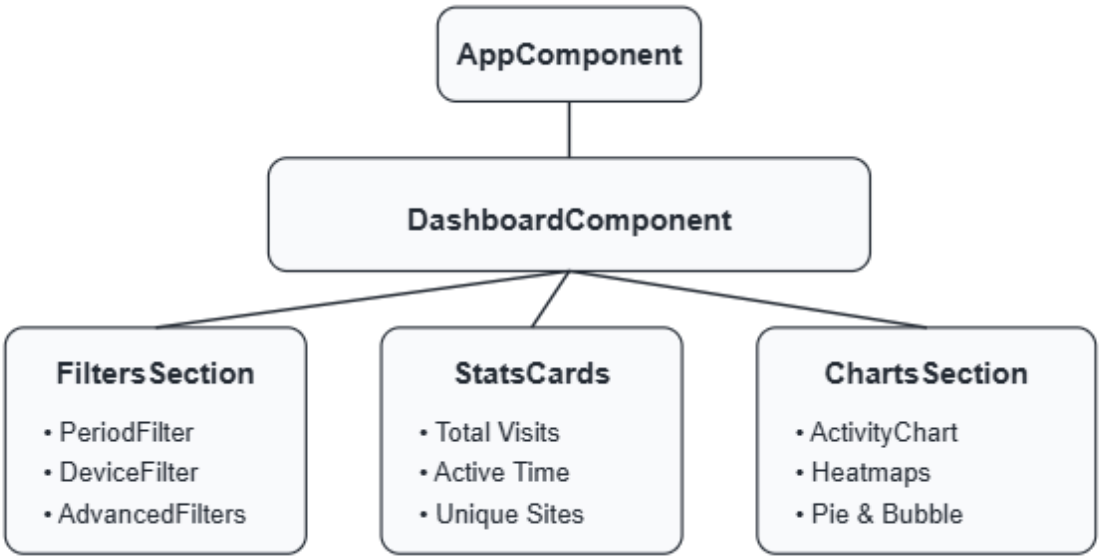


Рис. 3.2.1 - *Ієрархія компонентів Frontend додатку*

3.2.2. Компонент Dashboard

Основний компонент DashboardComponent координує роботу всього додатку:

Основні методи:

- loadData() - завантаження даних з API
- decryptHistoryData() - дешифрування даних історії
- updateCharts() - оновлення графіків на основі завантажених даних
- applyFilters() - застосування фільтрів до даних
- searchSites() - пошук по сайтах

Життєвий цикл компонента Dashboard розпочинається з його **ініціалізації (ngOnInit)**, під час якої виконується перевірка автентифікації користувача, здійснюється підписка на зміни стану автентифікації, а також ініціалізація початкових значень .

Далі відбувається **отримання ключа шифрування**: компонент спершу намагається отримати ключ із localStorage, а якщо ключ не знайдено, він надсилає запит до Extension через chrome.runtime.sendMessage і зберігає отриманий ключ для подальшого використання.

Наступним кроком є **завантаження даних з API**: компонент викликає метод loadData() як при ініціалізації, так і при зміні фільтрів, використовуючи **RxJS** для ефективної обробки асинхронних операцій та відображення повідомлень про помилки .

Після успішного завантаження відбувається **дешифрування даних**: компонент проходить по всіх записах історії, викликаючи метод дешифрування для кожного запису окремо, після чого розшифровані дані додаються в спеціальне поле decryptedData .

Далі слідує **обробка та агрегація даних**, необхідна для візуалізації: дані групуються по днях для **Activity Chart**, створюються масиви для **теплових карт**, виконується категоризація сайтів для **Pie Chart** та розрахунок статистики для **Top Sites** .

Завершальний етап циклу — **оновлення графіків та статистики**: дані передаються до дочірніх компонентів через декоратор @Input, ініціюється тригер **Change Detection** для оновлення UI, а також оновлюються статистичні картки .



Рис. 3.2.2 - Діаграма життєвого циклу Dashboard компонента

3.2.3. Компоненти графіків

Компоненти графіків є окремими standalone компонентами, кожен з яких відповідає за відображення конкретного типу візуалізації. Всі компоненти використовують бібліотеку ApexCharts для рендерингу графіків.

Спільні принципи реалізації:

- Всі компоненти використовують @Input() для отримання даних
- Використання ngOnChanges() для реагування на зміни даних
- Оптимізація через ChangeDetectionStrategy.OnPush
- Підтримка темної теми через динамічне оновлення кольорів

Activity Chart - лінійний графік активності за днями:

- Відображає кількість відвідувань по днях у вигляді лінійного графіка
- Підтримує масштабування (zoom) та прокрутку (pan) для детального перегляду
- Адаптується до вибраного періоду (автоматичне масштабування осей)

- Інтерактивні tooltip з детальною інформацією про кожну точку
- Підтримка темної теми з адаптацією кольорів

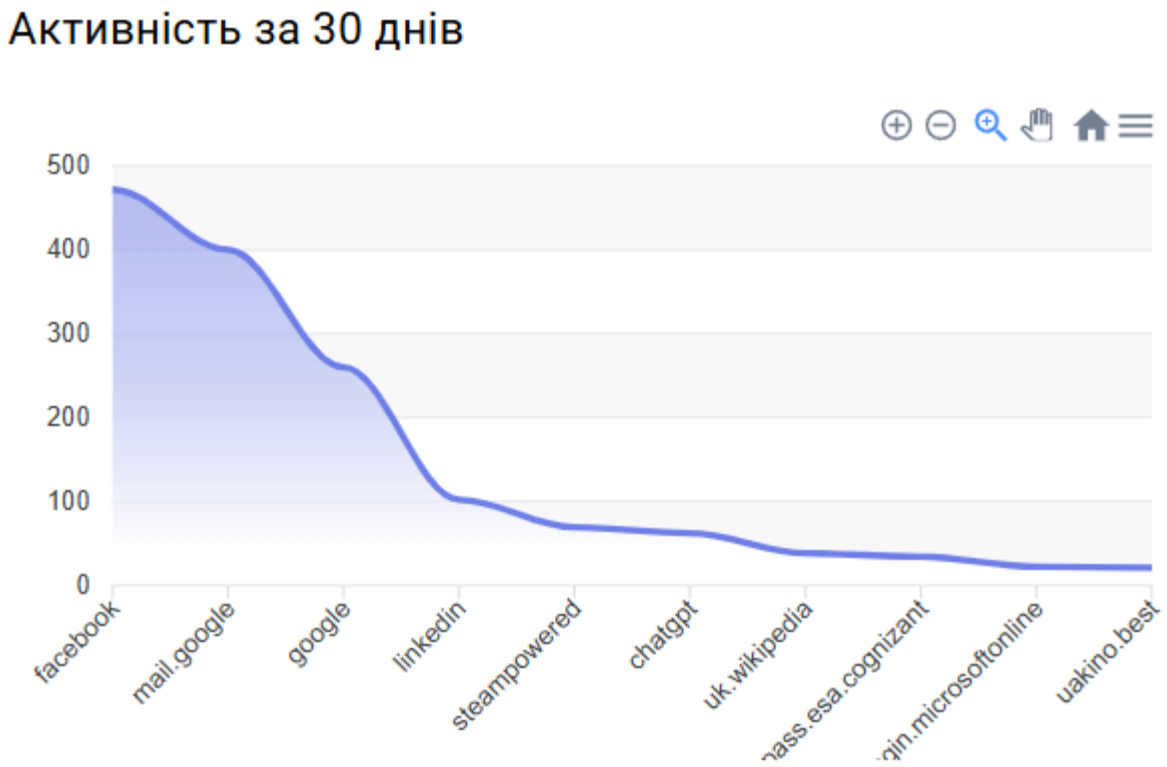


Рис. 3.2.3.1 - Лінійний графік активності

Activity Heatmap - теплова карта активності по днях тижня:

- Кожна комірка відображає активність в конкретний день тижня та годину
- Інтенсивність кольору залежить від кількості відвідувань (від світлого до темного)
- Підтримує інтерактивність (показує деталі при наведенні курсора)
- Використовує градієнт кольорів для візуального представлення інтенсивності
- Дозволяє визначити патерни активності (найактивніші дні та години)

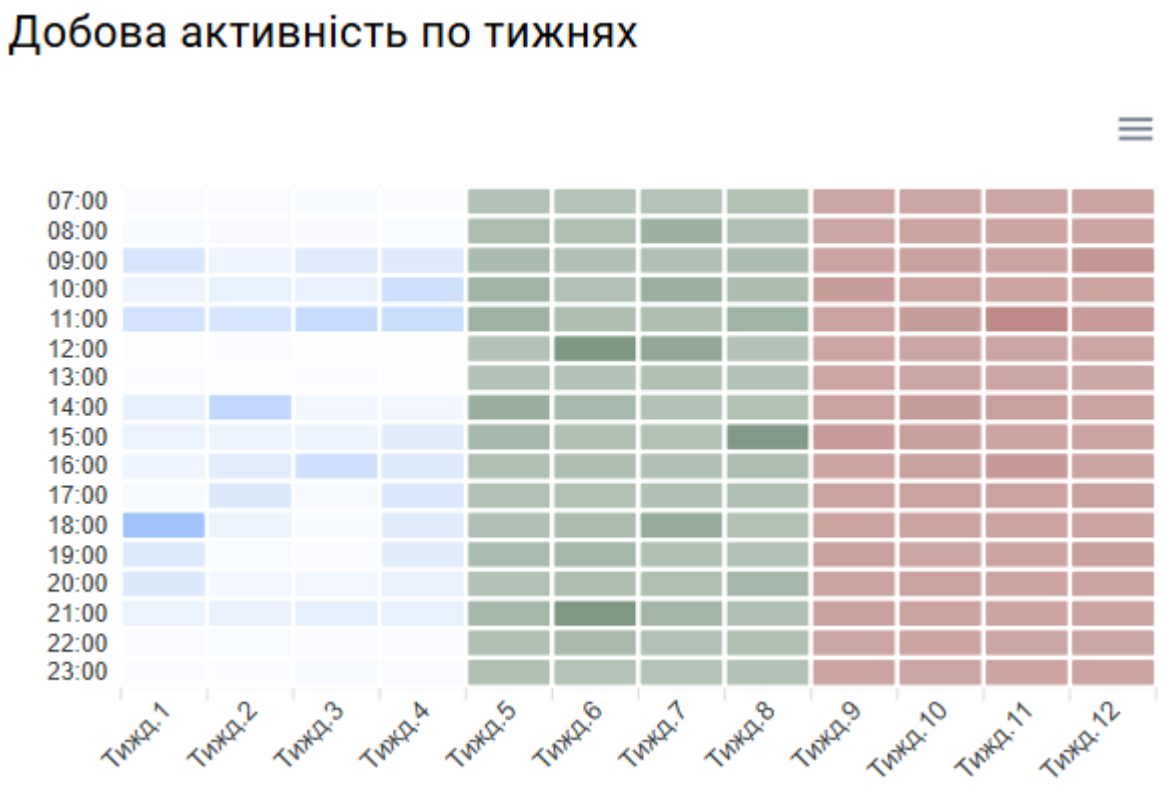


Рис. 3.2.3.2 - Приклад теплової карти активності

Time Heatmap - активність по годинах по тижнях:

- Горизонтальна вісь - години дня (0-23)
- Вертикальна вісь - тижні - Відображає патерни активності впродовж дня

Pie Chart - кругова діаграма розподілу по категоріях. Автоматична категорізація сайтів на основі домену:

- **Social** - соціальні мережі (Facebook, Twitter, Instagram, LinkedIn)
- **Work** - робочі сайти (GitHub, Stack Overflow, LinkedIn)
- **Entertainment** - розваги (YouTube, Netflix, Twitch)
- **News** - новинні сайти
- **Education** - навчальні ресурси

- **Other** - інші сайти
- Відображення частки кожної категорії у відсотках та часі
- Інтерактивні підказки з деталями при наведенні на сегмент
- Можливість виділення сегментів для акцентування уваги

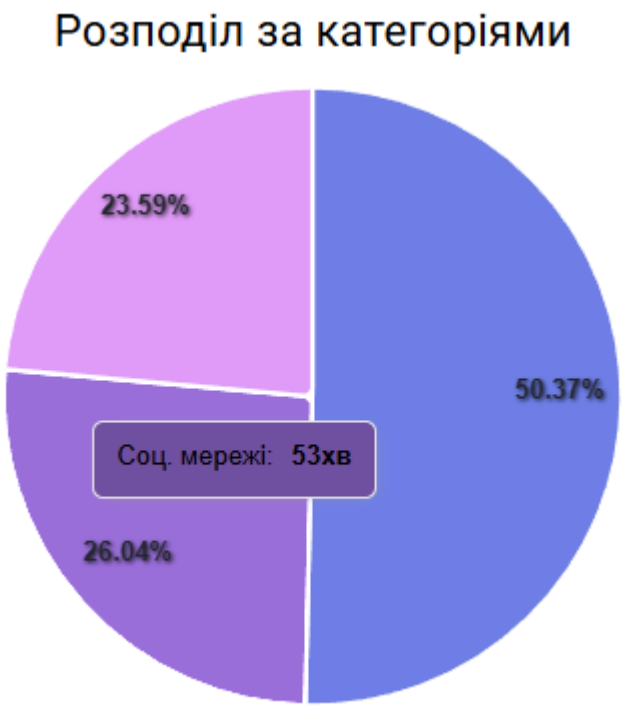


Рис. 3.2.3.3 - Приклад кругової діаграми

Bubble Chart - бульбашкова діаграма сайтів:

- Тривимірна візуалізація, де кожна бульбашка представляє сайт
- Вісь X - загальний час проведений на сайті (хвилини)
- Вісь Y - кількість відвідувань
- Розмір бульбашки - середній час на одне відвідування
- Колір бульбашки може відображати категорію сайту

- Дозволяє візуально ідентифікувати найактивніші сайти (великі бульбашки в правому верхньому куті)
- Підтримка масштабування для перегляду сайтів з невеликою активністю

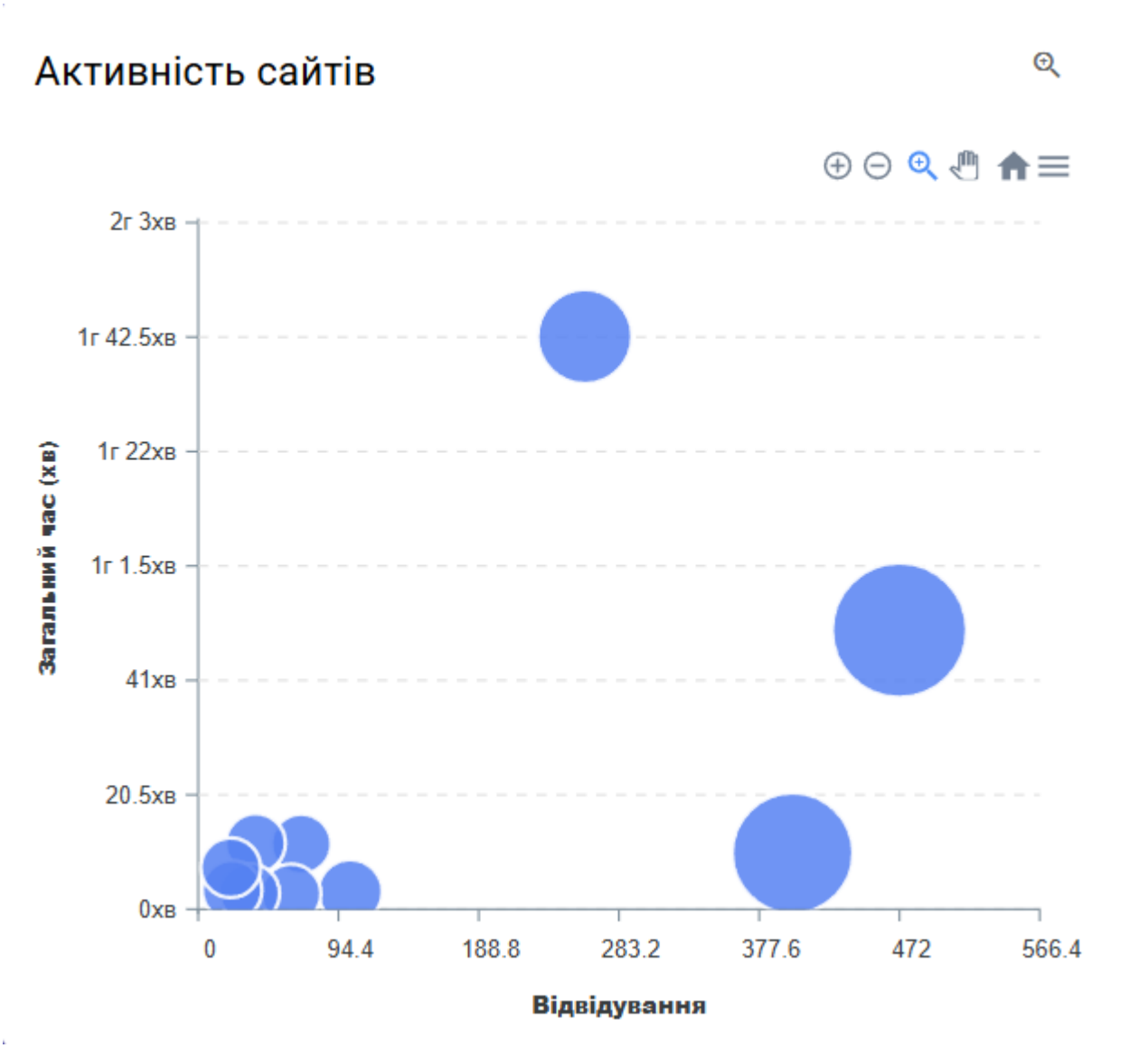


Рис. 3.2.3.4 - Приклад бульбашкової діаграми

Top Sites - список найпопулярніших сайтів:

- Сортування по кількості відвідувань або часу
- Відображення метрик для кожного сайту

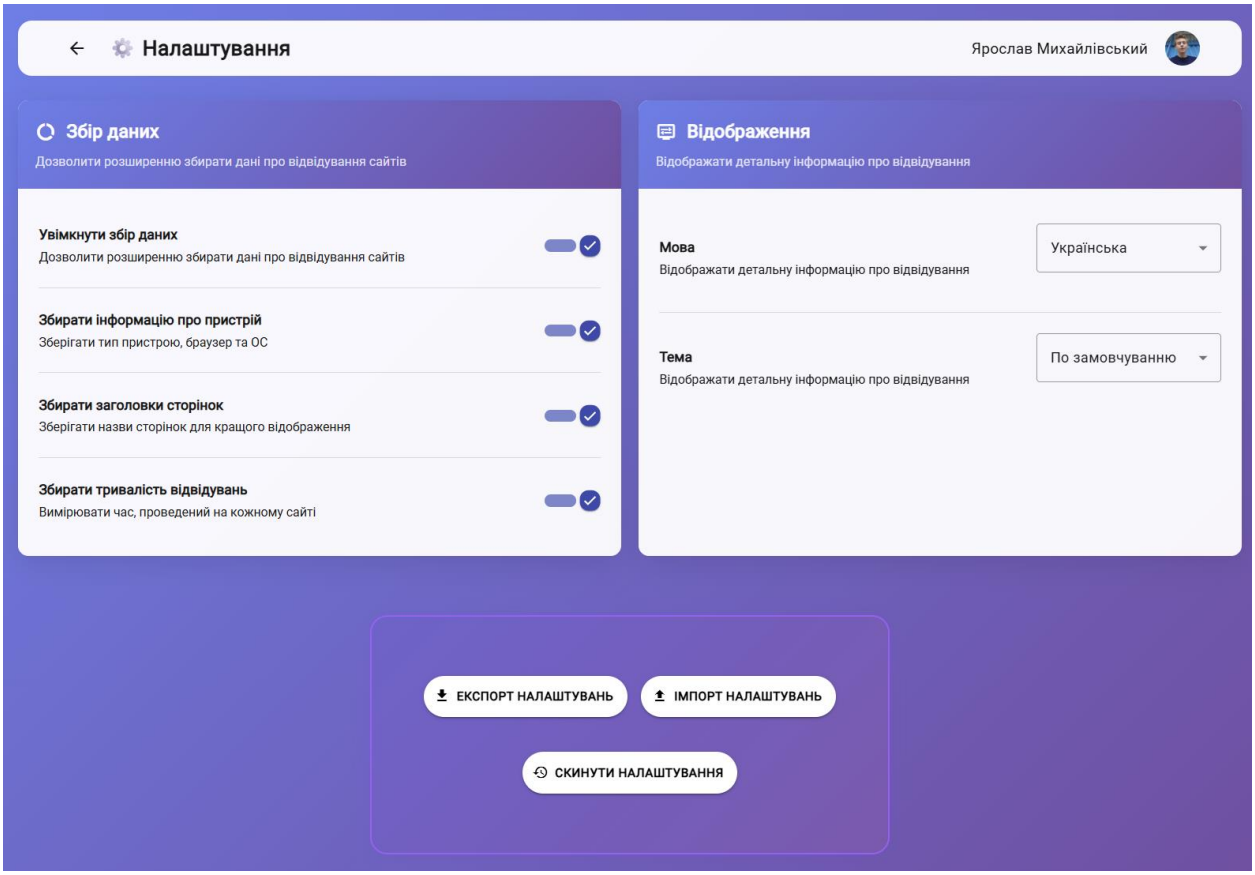


Рис. 3.2.3.5 - Зовнішній вигляд інтерфейсу налаштувань

3.2.4. Компонент Settings

Компонент налаштувань надає користувачеві повний контроль над роботою системи та її відображенням. Він дозволяє увімкнути або вимкнути збір даних та налаштувати детальні параметри збору (зокрема, тривалість та метадані).

Користувач може персоналізувати інтерфейс, вибравши мову інтерфейсу (українська/англійська) та тему (світла, темна або автоматична). Крім того, компонент дозволяє налаштувати приватність, включаючи термін зберігання даних та опціональну участь у глобальній аналітиці, а також здійснювати управління пристроями.

3.2.5. Сервіс ApiService

Сервіс ApiService інкапсулює всю логіку взаємодії з Backend API:

Основні методи:

- getHistory() - отримання історії з фільтрами
- getHistoryWithDecryptedData() - отримання історії з автоматичним дешифруванням
- getUserStatsWithDecryptedData() - отримання статистики з дешифрованими даними
- getDevices() - список пристроїв
- decryptData() - дешифрування окремих записів

Дешифрування даних: Сервіс автоматично дешифровує дані при їх отриманні:

1. Отримання ключа шифрування з localStorage або chrome.storage.local
2. Для кожного запису викликається decryptData()
3. Розшифровані дані додаються в поле decryptedData

3.3. Реалізація Chrome Extension

Chrome Extension є ключовим компонентом системи, який відповідає за збір даних про браузерну активність. Реалізація базується на Manifest V3, що забезпечує сучасний підхід до розробки розширень з підвищеною безпекою.

3.3.1. Теоретичні аспекти реалізації Extension

Service Worker архітектура в Manifest V3 відрізняється від Background Pages у V2. Service Worker активується за потреби та автоматично деактивується після завершення обробки подій. Це покращує продуктивність та зменшує споживання ресурсів [2].

Обробка подій виконується асинхронно через event listeners. Коли виникає подія (наприклад, відвідування сторінки), Service Worker активується, обробляє подію та може знову деактивуватися.

Зберігання даних виконується через chrome.storage.local, який забезпечує персистентне зберігання даних між сесіями. На відміну від localStorage, chrome.storage.local асинхронний та доступний в Service Worker.

3.3.2. Background Script (Service Worker)

Файл background.js містить головну логіку Extension:

Ініціалізація:

```
// Отримання або створення deviceId
deviceId = await getOrCreateStableDeviceId();

// Отримання або створення ключа шифрування
encryptionKey = await getOrCreateEncryptionKey();

// Генерація sessionId
sessionId = CryptoUtils.generateSessionId();
```

Збір історії:

```
chrome.history.onVisited.addListener(async (historyItem) => {
  if (!isCollecting) return;

  // Збір метаданих
  const metadata = {
    visitTime: historyItem.lastVisitTime,
    visitCount: historyItem.visitCount,
```

```
    typedCount: historyItem.typedCount
  };

  // Шифрування приватних даних
  const privateData = {
    url: historyItem.url,
    title: historyItem.title
  };
  const encryptedData = CryptoUtils.encrypt(privateData, encryptionKey);

  // Створення хешу домену
  const domainHash = CryptoUtils.hashDomain(historyItem.url);

  // Відправка на сервер
  await sendToBackend({
    encryptedData,
    domainHash,
    ...metadata
  });
});
```

3.3.3. Реалізація шифрування в Extension

Клас CryptoUtils в crypto-utils.js надає методи шифрування:

Шифрування:

```
static encrypt(data, key) {
  const dataStr = JSON.stringify(data);
  const encoded = this.utf8ToBase64(dataStr + '|' + key);
```

```
    return encoded;
  }
```

Дешифрування:

```
  static decrypt(encryptedData, key) {
    const decrypted = this.base64ToUtf8(encryptedData);
    const parts = decrypted.split('|');
    if (parts[1] === key) {
      return JSON.parse(parts[0]);
    }
    throw new Error('Invalid key');
  }
```

Безпека реалізації в Extension забезпечується через використання криптографічно безпечних методів для генерації ключів. Реалізований захист від timing attacks (атак за часом) шляхом підтримання постійного часу виконання операцій [12]. Перед обробкою даних проводиться валідація формату зашифрованих даних. Крім того, важливою частиною безпеки є ретельна обробка помилок, яка не розкриває деталей про справжню причину виникнення проблеми зловмисникам.

3.3.4. Оптимізація відправки даних

Extension відправляє дані пакетами для оптимізації продуктивності та зменшення навантаження на сервер:

Стратегія батч-відправки:

1. **Ефективна обробка**
   - Збір даних в буфер при їх надходженні
   - Буфер має обмежений розмір (наприклад, 100 записів)

2. **Умови відправки**
    - Відправка при досягненні розміру буфера
    - Відправка через заданий інтервал часу (наприклад, кожні 30 секунд)
    - Відправка при закритті Extension (якщо є невідправлені дані)
3. **Ефективна обробка**
    - Використання POST /api/history/batch для одночасної обробки багатьох записів
    - Обробка помилок та повторні спроби при збоях
    - Логування результатів відправки для діагностики

Стратегія батч-відправки даних з Chrome Extension приносить низку значних переваг. Вона сприяє зменшенню загальної кількості HTTP-запитів , що автоматично призводить до підвищення продуктивності сервера і одночасно зменшує навантаження на мережу. Крім того, така оптимізація позитивно впливає на клієнтську сторону, забезпечуючи економію заряду батареї на мобільних пристроях.

3.3.5. Реалізація автентифікації в Extension

Extension використовує Chrome Identity API для авторизації через Google OAuth 2.0. Це дозволяє безпечно автентифікувати користувача без необхідності вводити пароль безпосередньо в Extension.

Процес автентифікації:

1. **Запит токена через Chrome Identity API**

```
chrome.identity.getAuthToken({ interactive: true }, async (token) => {
  // token - це access token від Google
});
```

2. **Отримання idToken від Google**
   - Використання access token для отримання idToken
   - idToken містить інформацію про користувача та підпис Google
3. **Передача токена до Frontend**
   - Відправка idToken через message passing
   - Frontend використовує idToken для входу через Firebase Auth
4. **Збереження стану автентифікації**
   - Збереження інформації про користувача в chrome.storage
   - Використання токена для авторизації API запитів

Безпека токенів автентифікації забезпечується кількома ключовими механізмами. Токени зберігаються безпечно через Chrome Identity API, що мінімізує ризики їх компрометації. Система також гарантує автоматичне оновлення токенів при закінченні терміну їхньої дії, підтримуючи безперервну та безпечну роботу користувача. Крім того, передбачена ретельна обробка помилок автентифікації, що є важливим для захисту від потенційних загроз безпеки.

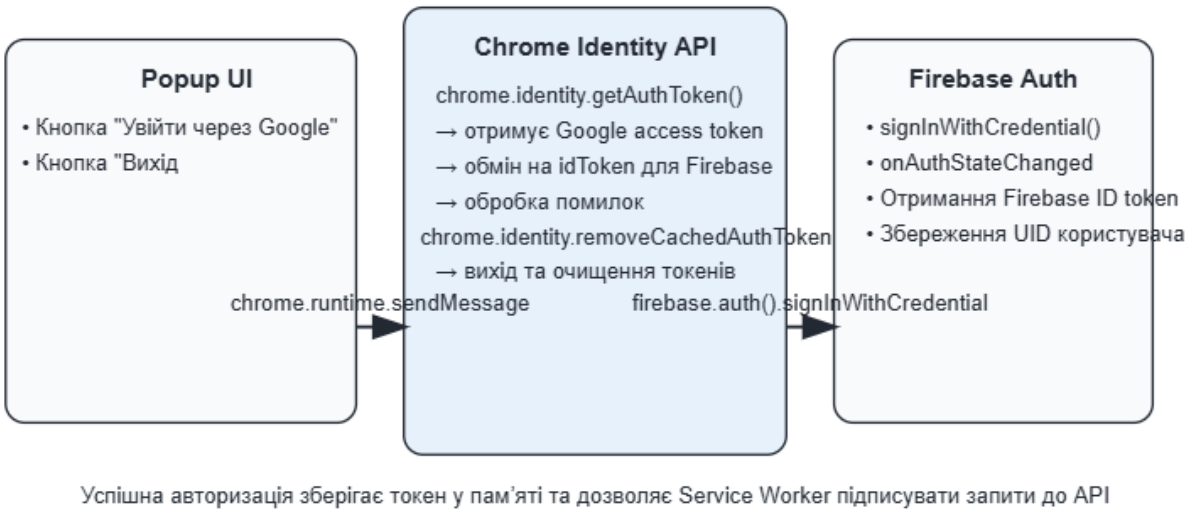


Рис. 3.3.5 - Схема автентифікації в Extension

3.4. Реалізація візуалізації даних

Візуалізація даних є одним із ключових аспектів системи, оскільки вона дозволяє користувачам інтуїтивно сприймати великі обсяги інформації, і її реалізація базується на сучасних бібліотеках та практиках.

Для створення графіків та діаграм обрано бібліотеку **ApexCharts**. Ця сучасна бібліотека забезпечує великий вибір типів графіків (понад 10 різних типів) , високу інтерактивність та анімації для покращення UX , підтримку темної теми , адаптивність (автоматичне масштабування під різні розміри екранів) , а також високу продуктивність завдяки використанню SVG замість Canvas [6].

Перед відображенням дані проходять складну обробку, яка перетворює сирі дані історії у формат, зручний для візуалізації, оскільки кожен тип графіка має свої специфічні вимоги. Загальні принципи цієї обробки включають **агрегацію** (групування даних за критеріями: день, година, категорія), **фільтрацію** (видалення нерелевантних даних або даних, що не відповідають вибраним фільтрам), **сортування** (впорядкування даних для найкращого відображення), **нормалізацію** (приведення даних до єдиного формату) та **оптимізацію** (обмеження кількості точок даних для підвищення продуктивності).

Перед відображенням дані проходять обробку:

Activity Chart:

```
// Групування по днях
const groupedByDay = historyItems.reduce((acc, item) => {
  const day = formatDate(item.visitTime);
  acc[day] = (acc[day] || 0) + item.visitCount;
  return acc;
}, {});
```

```
// Конвертація в формат ApexCharts
const chartData = Object.entries(groupedByDay).map(([x, y]) => ({ x, y }));
```

Heatmap:

```
// Групування по днях тижня та годинах
const heatmapData = historyItems.reduce((acc, item) => {
  const dayOfWeek = item.visitTime.getDay();
  const hour = item.visitTime.getHours();
  const key = `${dayOfWeek}-${hour}`;
  acc[key] = (acc[key] || 0) + item.visitCount;
  return acc;
}, {});

// Конвертація в формат ApexCharts Heatmap
const series = Array.from({ length: 7 }, (_, day) => ({
  name: getDayName(day),
  data: Array.from({ length: 24 }, (_, hour) => ({
    x: hour,
    y: heatmapData[`${day}-${hour}`] || 0
  }))
}));
```

Pie Chart:

```
// Категорізація сайтів
const categorized = historyItems.reduce((acc, item) => {
  const category = categorizeWebsite(item.decryptedData.url);
  acc[category] = (acc[category] || 0) + item.duration;
  return acc;
```

```
}, {});

// Конвертація в формат ApexCharts Pie
const pieData = Object.entries(categorized).map(([label, value]) => ({
  label,
  value
}));
```

3.4.3. Алгоритм категорізації сайтів

Функція categorizeWebsite() реалізує інтелектуальну категорізацію сайтів на основі аналізу домену. Це дозволяє автоматично класифікувати сайти без необхідності ручного введення користувачем.

Алгоритм категорізації:

5. **Витягування домену з URL**
   - Парсинг URL та витягування основного домену
   - Видалення www префіксу
   - Нормалізація домену (приведення до нижнього регістру)
6. **Перевірка по словниках категорій**
   - Кожна категорія має словник відомих доменів
   - Перевірка наявності домену в словниках
   - Пріоритет перевірки категорій (більш специфічні категорії перевіряються першими)
7. **Перевірка по ключових словах**
   - Аналіз домену на наявність ключових слів
   - Наприклад, домени з “news”, “newspaper” → News
   - Домени з “shop”, “store”, “market” → Shopping

8. **Категорії сайтів:**

   – **Social** - соціальні мережі та платформи для спілкування:
     • Facebook, Twitter, Instagram, LinkedIn, Reddit, TikTok, Telegram
   – **Work** - робочі та професійні ресурси:
     • LinkedIn, GitHub, Stack Overflow, GitLab, Jira, Confluence, Slack, Microsoft Teams
   – **Entertainment** - розваги та медіа:
     • YouTube, Netflix, Twitch, Spotify, SoundCloud, TikTok, Instagram
   – **News** - новинні та інформаційні сайти:
     • Сайти з ключовими словами: news, newspaper, magazine, media
   – **Shopping** - інтернет-магазини та торгові платформи:
     • Amazon, eBay, Rozetka, Prom.ua, сайти з ключовими словами: shop, store, market
   – **Education** - навчальні ресурси:
     • Coursera, Udemy, Khan Academy, Wikipedia, освітні заклади (.edu домени)
   – **Other** - інші сайти, які не підпадають під попередні категорії

9. **Повернення результату**

   – Якщо категорія знайдена - повертається назва категорії
   – Якщо не знайдена - повертається “Other”

Розширюваність:

- Словники категорій легко розширюються новими доменами
- Можливість додавання користувацьких правил категорізації
- Навчання на основі попередніх класифікацій користувача

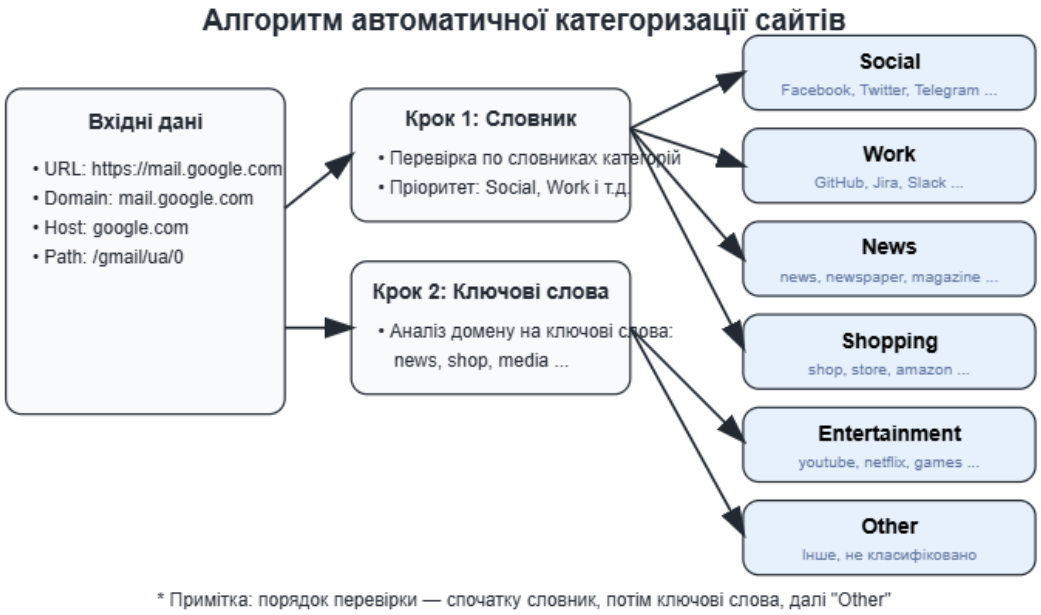


Рис. 3.4.3 - Схема алгоритму категорізації

РОЗДІЛ 4. ТЕСТУВАННЯ ТА РОЗГОРТАННЯ

Розділ містить опис стратегії тестування системи, інструкції з встановлення та налаштування, а також методи розгортання на production середовище.

4.1. Стратегія тестування

Тестування є критично важливою частиною розробки системи, оскільки забезпечує надійність, безпеку та коректність роботи всіх компонентів. Стратегія тестування охоплює різні типи тестів для різних аспектів системи.

4.1.1. Теоретичні основи тестування

Unit Testing - тестування окремих функцій та методів у ізоляції. Дозволяє перевірити коректність логіки на рівні найменших компонентів.

Integration Testing - тестування взаємодії між різними компонентами системи. Перевіряє, чи правильно компоненти працюють разом.

End-to-End Testing - тестування повного потоку операцій від початку до кінця. Симулює реальне використання системи користувачем.

Security Testing - спеціальне тестування для перевірки безпеки системи, включаючи тестування на вразливості та перевірку шифрування.

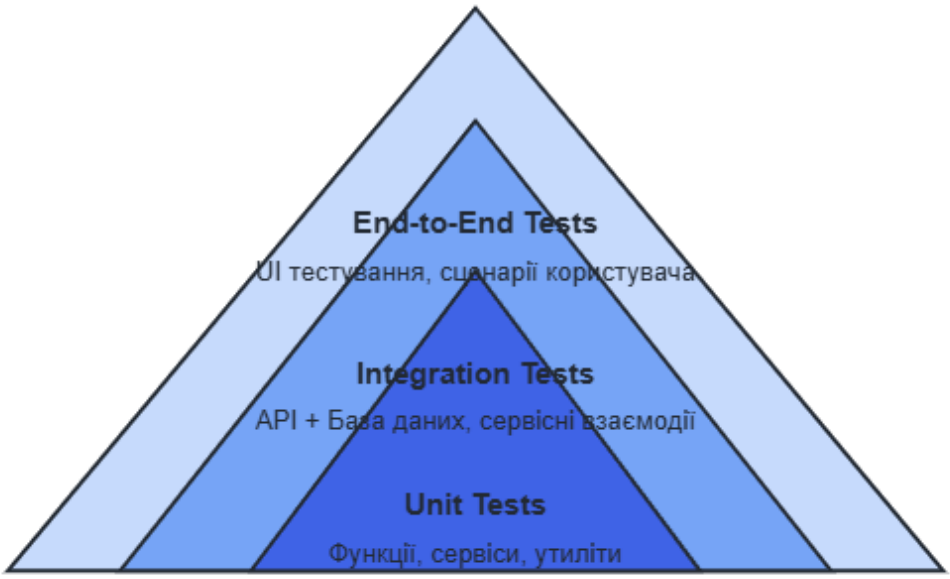


Рис. 4.1.1 - Піраміда тестування

4.1.2. Типи тестування

У системі Chartex було застосовано кілька ключових типів тестування для забезпечення її якості, безпеки та продуктивності.

Функціональне тестування зосереджувалося на перевірці коректності роботи всіх бізнес-функцій. Воно включало перевірку збору даних Extension, повну перевірку процесів шифрування/дешифрування, тестування всіх API endpoints, а також перевірку коректності візуалізації графіків та роботи фільтрів і пошуку.

Тестування безпеки було критично важливим, враховуючи архітектуру Zero Knowledge. Воно передбачало поглиблену перевірку шифрування даних, надійності автентифікації та авторизації кінцевих точок. Крім того, проводилася перевірка захисту від SQL injection (щодо MongoDB injection) та коректності CORS налаштувань.

Тестування продуктивності було спрямоване на оцінку швидкості роботи системи, включаючи перевірку швидкості завантаження даних, продуктивності обробки великих обсягів даних та оптимізації запитів до бази даних. Нарешті,

тестування сумісності забезпечувало коректну роботу системи в різних версіях Chrome, на різних операційних системах та різних пристроях.

4.1.3. Тестування вручну

Збір даних:

1. Встановити Extension
2. Перевірити авто++матичний збір при відвідуванні сайтів
3. Перевірити збір метаданих
4. Перевірити шифрування перед відправкою

Backend API:

1. Перевірити всі endpoints через Postman/curl
2. Перевірити автентифікацію
3. Перевірити валідацію даних
4. Перевірити обробку помилок

Frontend:

1. Перевірити відображення всіх графіків
2. Перевірити фільтри та пошук
3. Перевірити дешифрування даних
4. Перевірити адаптивність інтерфейсу

4.2. Інструкції з встановлення

Встановлення системи Chartex вимагає налаштування кількох компонентів: Backend сервера, Frontend додатку та Chrome Extension. Кожен компонент має свої вимоги та інструкції.

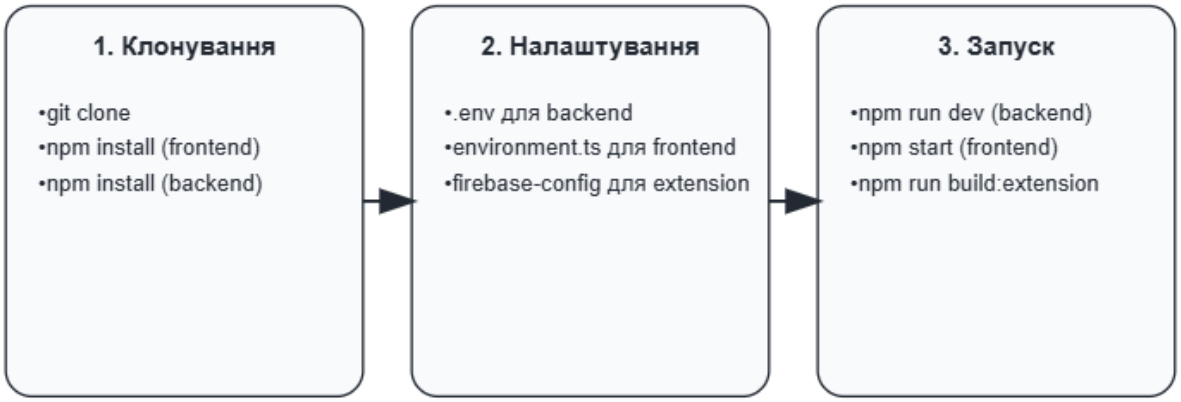


Рис. 4.2 - Схема процесу встановлення

4.2.1. Вимоги до системи

Програмне забезпечення:

- Node.js 18 або вище
- JavaScript runtime для Backend та Frontend
- npm або yarn - менеджер пакетів для встановлення залежностей
- MongoDB 6 або вище - база даних для зберігання даних
- Chrome браузер версії 88+ (для Extension, підтримка Manifest V3)
- Git - система контролю версій для клонування репозиторію

Апаратні вимоги:

- Мінімум 4 GB RAM
- 500 MB вільного місця на диску
- Інтернет-з’єднання для завантаження залежностей та роботи з API

4.2.2. Встановлення Backend

```
# Клонувати репозиторій
git clone <repository-url>
cd chartex/backend

# Встановити залежності
npm install

# Створити .env файл
cp env.example .env

# Налаштувати змінні середовища в .env:
# PORT=5000
# MONGODB_URI=mongodb://localhost:27017/chartex
# FIREBASE_PROJECT_ID=your-project-id
# FIREBASE_CLIENT_EMAIL=your-email@project.iam.gserviceaccount.com
# FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"

# Запустити MongoDB (якщо локально)
mongod

# Запустити сервер
npm run dev  # для розробки
```

```
# або
npm start    # для production
```

4.2.3. Встановлення Frontend

```
cd frontend

# Встановити залежності
npm install

# Налаштувати environment файли
# frontend/src/environments/environment.ts (для розробки)
# frontend/src/environments/environment.prod.ts (для production)

# Запустити dev server
npm start  # Запускає на http://localhost:4200

# Або зібрати для production
npm run build:prod
```

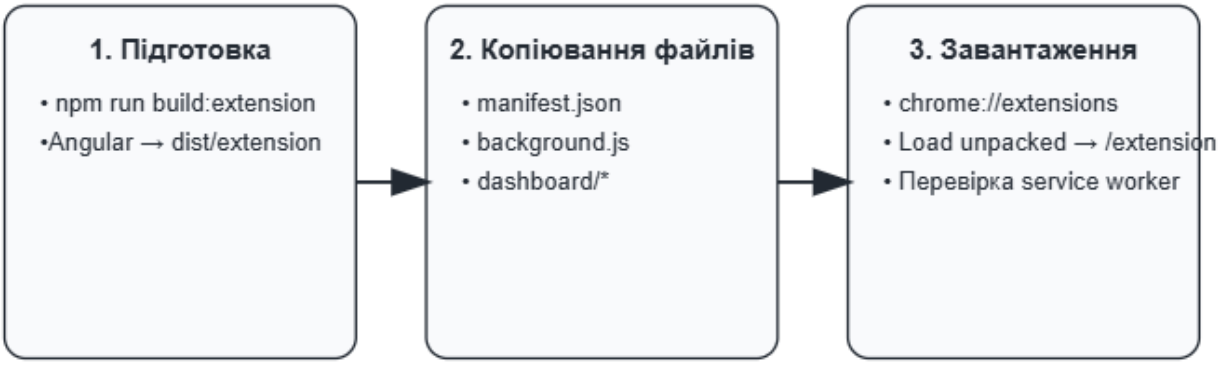


Рис. 4.2.3 - Процес збірки Extension

4.2.4. Встановлення Extension

```
# Спочатку зібрати frontend для extension
cd frontend
npm run build:extension

# Або використати скрипт
.\build-extension.bat  # Windows
# або
.\build-extension.ps1  # PowerShell

# Після збірки:
# 1. Відкрити Chrome
# 2. Перейти до chrome://extensions/
# 3. Увімкнути "Developer mode"
# 4. Натиснути "Load unpacked"
# 5. Вибрати папку extension/
```

4.3. Інструкції з налаштування

Після встановлення всіх компонентів необхідно виконати налаштування для їх коректної роботи. Налаштування включає конфігурацію Firebase, MongoDB, CORS та інших компонентів системи.

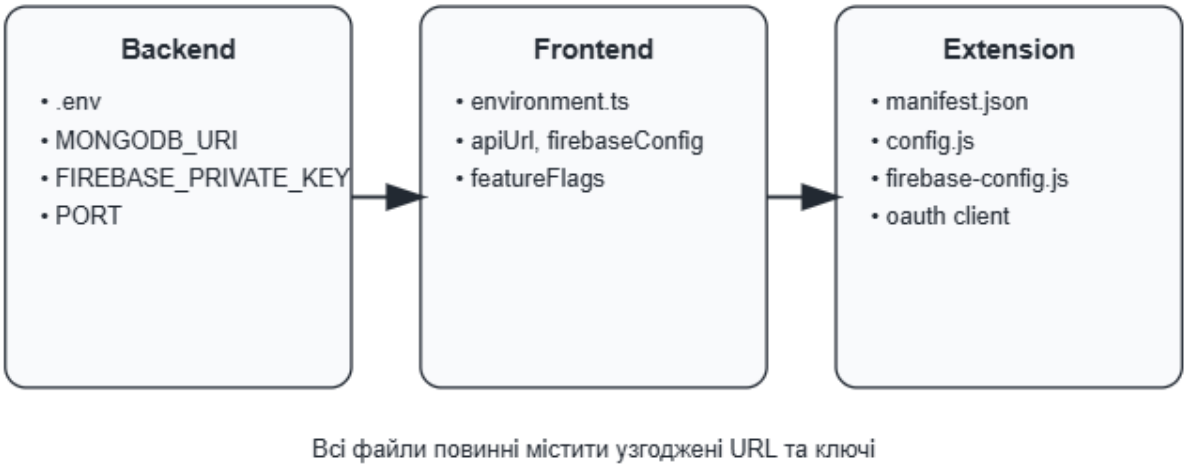


Рис. 4.3 - Схема налаштування системи

4.3.1. Налаштування Firebase

1. Створити проект в Firebase Console
2. Увімкнути Authentication з Google провайдером
3. Створити Service Account для Backend
4. Скачати приватний ключ
5. Додати ключ в .env файл Backend

4.3.2. Налаштування MongoDB

Локально:

```
# Запустити MongoDB
mongod

# Підключитися до MongoDB
mongosh

# Створити базу даних (автоматично при першому підключенні)
use chartex
```

MongoDB Atlas (Cloud):

1. Створити кластер в MongoDB Atlas
2. Отримати connection string
3. Додати IP адресу до whitelist
4. Оновити MONGODB_URI в .env

4.3.3. Налаштування CORS

У файлі backend/src/index.js налаштувати CORS для ваших доменів:

```
app.use(cors({
  origin: [
    'http://localhost:4200',        // Frontend dev
    'http://localhost:8080',        // Frontend prod (якщо інший порт)
    'https://your-domain.com',      // Production domain
    /^chrome-extension:\/\/.*$/     // Chrome Extension
  ],
  credentials: true
}));
```

4.3.4. Налаштування Extension

У файлі extension/config.js налаштувати URL API:

```
const config = {
  api: {
    development: 'http://localhost:5000/api',
    production: 'https://your-api-domain.com/api'
  },
```

```
  // ...
};
```

4.4. Розгортання

Розгортання системи на production середовище вимагає уважного підходу до конфігурації та безпеки. Розділ описує різні методи розгортання для різних компонентів системи.

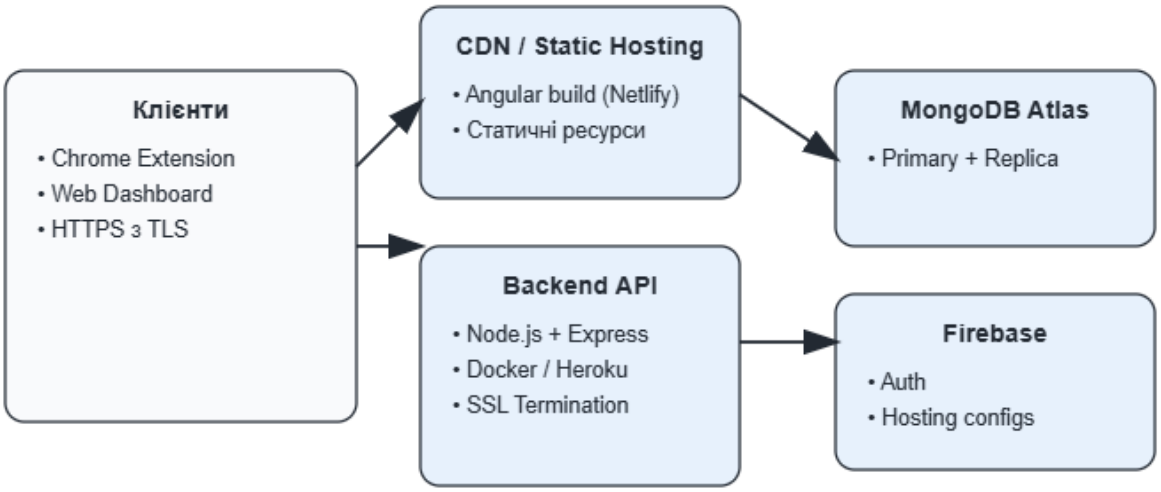


Рис. 4.4 - Архітектура production середовища

Для розгортання бекенду використовувалась наступна конфігурація:

Docker:

```
# Зібрати образ
docker build -t chartex-backend ./backend

# Запустити контейнер
docker run -d \
  -p 5000:5000 \
  -e MONGODB_URI=your-mongodb-uri \
```

```
  -e FIREBASE_PROJECT_ID=your-project-id \
  chartex-backend
```

Для фронтенду я використовував налаштування продемонстроване нижче:

Nginx:

```
server {
    listen 80;
    server_name your-domain.com;

    root /var/www/chartex-frontend;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Щоб опублікувати не менш важливу частину системи - chrome extension були здійснені наступні кроки:

1. Зібрати Extension: npm run build:extension
2. Створити ZIP архів з папки extension/
3. Завантажити в Chrome Web Store Developer Dashboard
4. Заповнити опис та скріншоти
5. Відправити на перевірку

ВИСНОВКИ

В ході виконання роботи було розроблено систему аналітики браузерної активності **Chartex**, яка забезпечує:

1. Проведено аналіз предметної області та визначено вимоги. Проведений аналіз існуючих рішень (вбудованої історії браузерів та сторонніх сервісів) виявив їхні основні недоліки: проблеми з приватністю, відсутність контролю користувача та обмежена функціональність. Це дало змогу сформулювати ключові вимоги до розробки системи Chartex.

2. Розроблено та впроваджено трирівневу архітектуру. Розроблена модульна архітектура системи Chartex (Chrome Extension, Backend API, Frontend Dashboard) базується на принципах розділення відповідальностей, забезпечуючи її масштабованість та надійність.

3. Реалізовано Backend компонент для безпечного зберігання даних. Створено серверний компонент на основі Node.js, Express.js та MongoDB, який відповідає за прийом, валідацію та зберігання зашифрованих даних історії користувача без доступу до ключа шифрування.

4. Реалізовано Frontend для інтерактивної візуалізації та аналітики. Створено клієнтський веб-інтерфейс (Dashboard) на Angular 19 з використанням ApexCharts, що відповідає за отримання зашифрованих даних, їхнє дешифрування на стороні клієнта та інтерактивне відображення аналітики.

5. Розроблено Chrome Extension для автоматичного збору даних. Реалізовано браузерне розширення на Manifest V3 зі Service Worker, яке здійснює автоматичний фоновий збір метаданих про активність, їхнє шифрування та батч-відправку на сервер.

6. Забезпечено максимальний захист приватності (Zero Knowledge). Забезпечена безпека даних через повну реалізацію Zero Knowledge архітектури, де всі приватні дані шифруються на клієнті за допомогою AES-256-GCM до відправки на сервер, а ключі шифрування зберігаються виключно локально.
7. Створено детальний інтерфейс візуалізації та фільтрації. Розроблений інтерфейс Dashboard надає детальну аналітику у вигляді різноманітних графіків (теплові карти, кругові діаграми) та пропонує розширену систему фільтрації та пошуку для глибокого аналізу браузерної активності.

Система демонструє, що можна створювати потужні інструменти аналітики, які водночас забезпечують максимальний захист приватності користувача через правильну архітектуру та використання сучасних технологій шифрування. Основні досягнення:

- Реалізована повноцінна трирівнева архітектура (Extension, Backend, Frontend) з чітким розділенням відповідальностей.
- Впроваджено Zero Knowledge шифрування для захисту приватних даних з використанням AES-256.
- Створено інтуїтивний інтерфейс з детальною візуалізацією через сучасні бібліотеки графіків.
- Забезпечено масштабованість та продуктивність системи через оптимізацію запитів та індексацію.
- Реалізовано автоматичну категорізацію сайтів для зручної аналітики.
- Забезпечено підтримку багатомовності та адаптивності інтерфейсу.

Система готова до використання та демонструє, що можна створювати потужні інструменти аналітики з максимальним захистом приватності. Система може бути розширена додатковими функціями, такими як:

- Експорт даних в різних форматах (CSV, JSON, PDF)
- Розширені фільтри та пошук
- Аналітика продуктивності та звичок
- Інтеграція з іншими сервісами

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Angular Documentation. [Електронний ресурс]. – Режим доступу: https://angular.io/docs
2. Chrome Extensions Documentation. [Електронний ресурс]. – Режим доступу: https://developer.chrome.com/docs/extensions/
3. Express.js Documentation. [Електронний ресурс]. – Режим доступу: https://expressjs.com/
4. Firebase Documentation. [Електронний ресурс]. – Режим доступу: https://firebase.google.com/docs
5. MongoDB Documentation. [Електронний ресурс]. – Режим доступу: https://www.mongodb.com/docs/
6. ApexCharts Documentation. [Електронний ресурс]. – Режим доступу: https://apexcharts.com/docs/
7. Zero Knowledge Architecture. [Електронний ресурс]. – Режим доступу: https://en.wikipedia.org/wiki/Zero-knowledge_architecture
8. GDPR Compliance Guide. [Електронний ресурс]. – Режим доступу: https://gdpr.eu/
9. AES Encryption Standard. [Електронний ресурс]. – Режим доступу: https://en.wikipedia.org/wiki/Advanced_Encryption_Standard
10. Node.js Documentation. [Електронний ресурс]. – Режим доступу: https://nodejs.org/docs/
11. Angular Material Documentation. [Електронний ресурс]. – Режим доступу: https://material.angular.io/

12. Crypto-JS Library. [Електронний ресурс]. – Режим доступу: https://cryptojs.gitbook.io/
13. RxJS Documentation. [Електронний ресурс]. – Режим доступу: https://rxjs.dev/
14. ngx-translate Documentation. [Електронний ресурс]. – Режим доступу: https://github.com/ngx-translate/core
15. Mongoose Documentation. [Електронний ресурс]. – Режим доступу: https://mongoosejs.com/docs/